

```
In [1]: # comments in Python start with a "#" and will not be executed - well commen
# Importing modules that will be necessary for the tutorial
import numpy as np # numpy is a general math/array manipulation library that
import matplotlib.image as mimg # matplotlib.image is used for displaying
import matplotlib.pyplot as plt # matplotlib.pyplot is used for general plot
import pandas as pd # pandas is a data manipulation library, we will use it
```

```
In [2]: def InlineImage(img, width, height, caption, fs):
# Author: BG
# Renders an image with a caption underneath using matplotlib
# Requires matplotlib.pyplot to be imported as plt and matplotlib.image to
# img - string, the location of an image file
# width, height - numbers, the dimensions of the figure the image is dis
# caption - string, the caption to be displayed beneath the image
# fs - number, the font size of the caption
fig, ax = plt.subplots(figsize=(width,height))
ax.imshow(mimg.imread(img))
ax.set_xlabel(caption, fontsize=fs)
ax.tick_params(
    which='both',
    bottom=False,
    top=False,
    left=False,
    labelleft=False,
    labelbottom=False)
ax.spines["top"].set_visible(False)
ax.spines["left"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.spines["bottom"].set_visible(False);
```

Final Project | RPM Motor Control ENPH 259

Objective

The objective of this final project is to construct a servo control loop circuit that uses feedback from the revolutions per minute of an electric motor to control its speed, using all the circuit components that were learned throughout the previous labs. The learning goals of this project is to integrate all the learned components from this course, as well as some new ones (phototransistor, DC motor), into a comprehensive circuit, applying skills learned throughout the course to analyze, test, and troubleshoot the circuit to create a working motor rpm-control circuit.

See figure 1 for a general overview of the circuit. A disc is attached to the motor with a single slit in it. When the motor is turned on the disc rotates and when the slit passes by the LED some light falls on a phototransistor, causing it to conduct thereby generating a clock(or CLK) pulse (every time the slit passes over the LED/

phototransistor). Next a counter counts the number of CLK pulses between RESET pulses. Just before the RESET pulse, an 8 bit D-latch stores the counter output (using the LATCH output) which is then converted from a digital signal (in binary) to an analog signal (i.e., an digital-to-analog convertor or DAC) using an R – 2R ladder network. The analog output of the DAC is then compared (subtracted from) to an analog voltage (representing the set point for speed of the motor) and integrated. The output of this operation is then used to control the current through the motor via power transistor.

In [3]: `InlineImage('wholeCircuit.png', 10, 10, 'Fig. 1: General Overview of Entire`

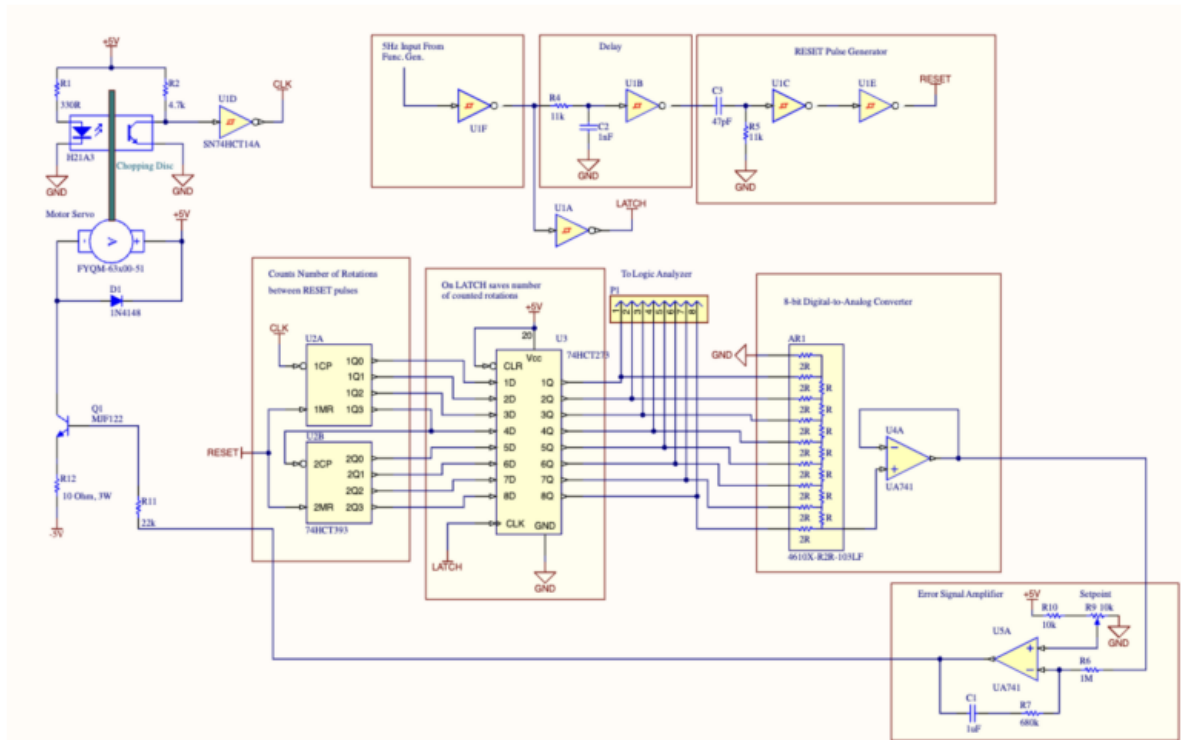


Fig. 1: General Overview of Entire Circuit

4.1 LATCH and RESET generator

In [4]: `InlineImage('74HC14pinout.png', 7, 7, 'Fig. 2: 74HC14N Hex Schmitt Trigger I`

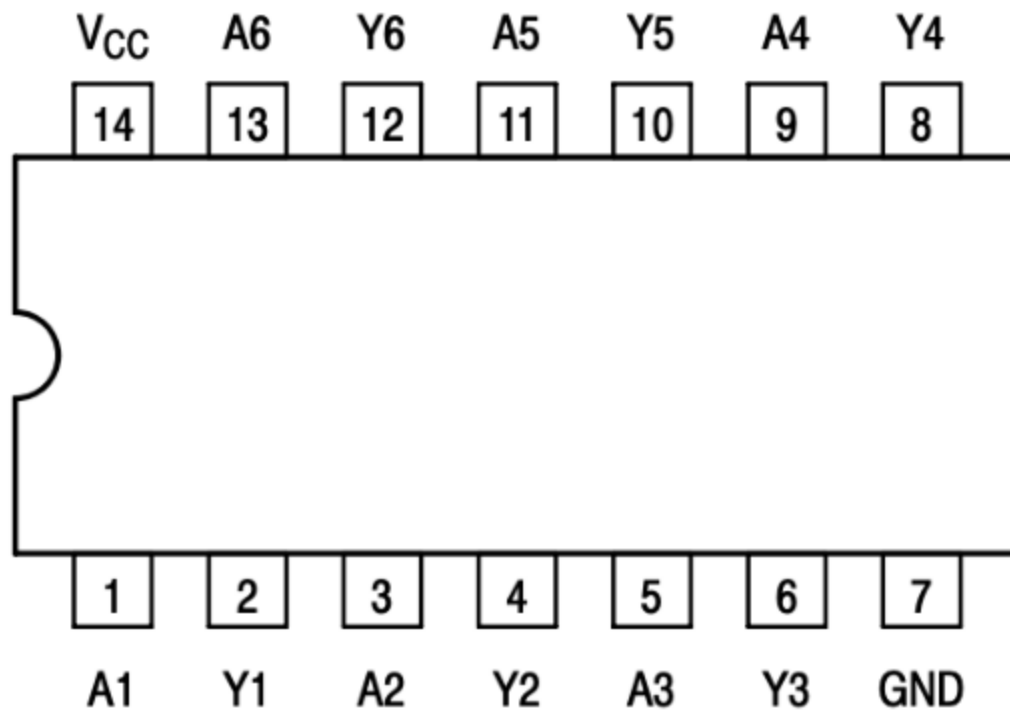


Fig. 2: 74HC14N Hex Schmitt Trigger Inverter Pinout

```
In [5]: InlineImage('74HC14NfuncTable.png', 5, 5, 'Fig. 3: 74HC14N Function Table',
```

FUNCTION TABLE

Inputs	Outputs
A	Y
L	H
H	L

Fig. 3: 74HC14N Function Table

```
In [6]: InlineImage('74HC14NlogicDiag.png', 6, 6, 'Fig. 4: 74HC14N Logic Diagram', 1
```

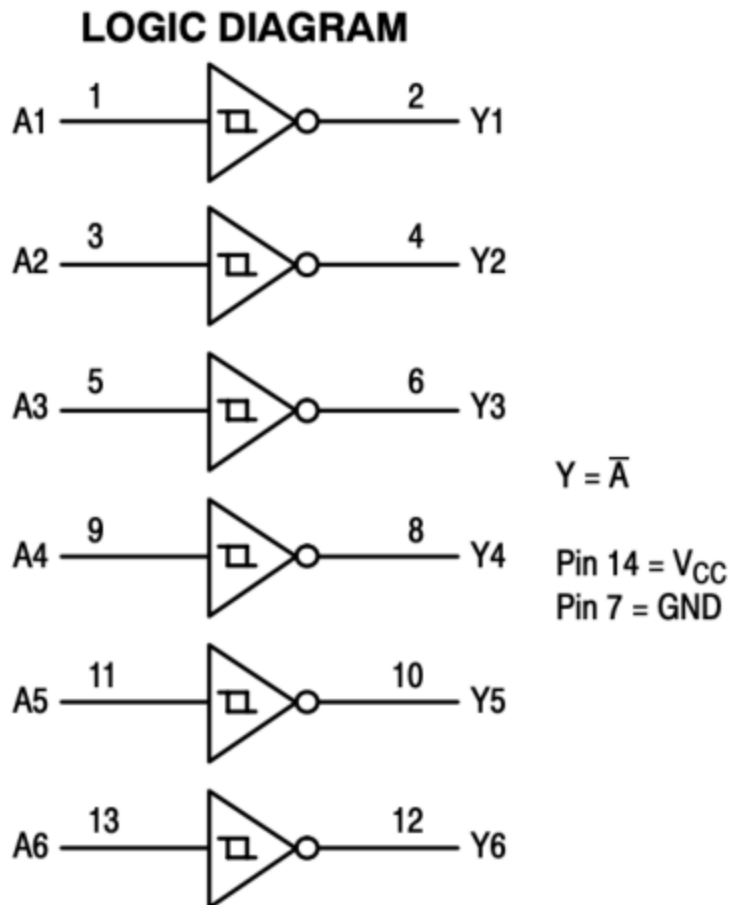


Fig. 4: 74HC14N Logic Diagram

First, I make the delay circuit. I start by really thinking about how I want to lay out this part of the circuit; i.e. which inverters I want to use for which parts of figure 2 on the instructions. I opt to use the output of the delay circuit to the latch (U1A) to be pin 2 on the 74HC14N, so that there is less wire cross-over. Similarly, I have the output of the reset on the side of the circuit board that opens up to the rest of the board. To reduce wiring I have U1F, U1B, and U1C from figure 2 of the instructions pdf all adjacent to each other (pins 8-13), and I have the resistors and capacitors very compactly together. See figure 5 for the delay circuit diagram, and 6 for the breadboard circuit.

In [7]: `InlineImage('delayCircDiag.jpeg', 11, 7, 'Fig. 5: Delay Circuit Diagram', 14`

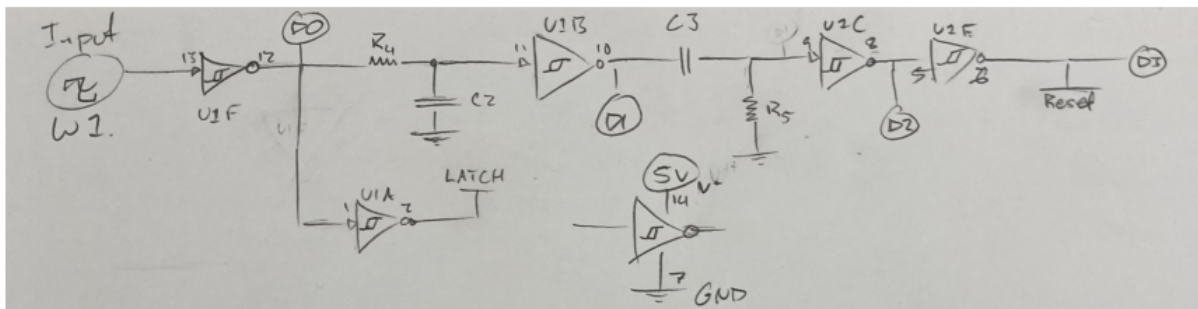


Fig. 5: Delay Circuit Diagram


```
In [8]: InlineImage('delayLabeled.jpeg', 10, 10, 'Fig. 6: Delay Circuit Breadboard',
```

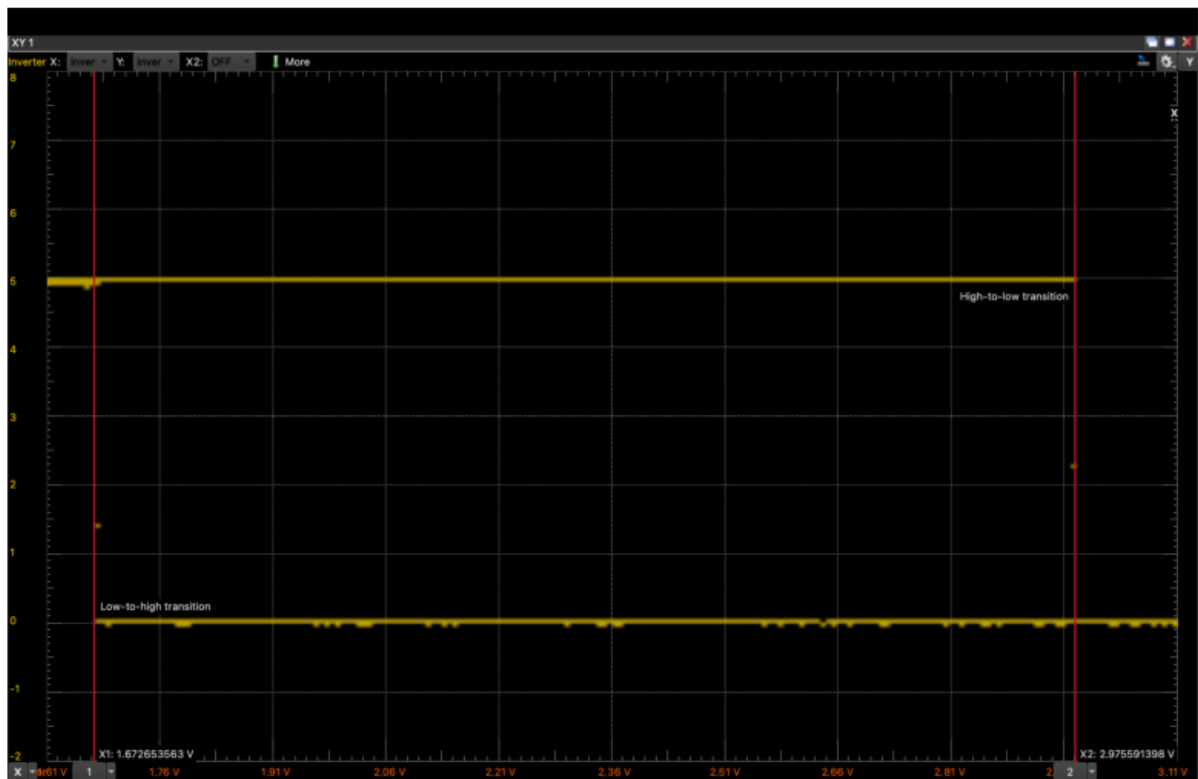


Fig. 7: Measured Hysteresis Plot

To see how a delay is introduced, let's go through the circuit sequentially. We have a square wave input coming in through the first inverter, which naturally inverts that square wave (high $\rightarrow$ low, low $\rightarrow$ high). That signal goes through another inverter to the latch (so that the latch input is basically just a squared up signal of what the input is (seeing that our input is a square wave, in a sense the double inverters aren't necessary for the latch output)). Now suppose that input signal transitions from high to low. Since the signal was high, the output of the inverter U1F was low, so that there was no potential difference across the resistor and capacitor. Hence, the capacitor is uncharged, and no current flows through the resistor, with the inverter U1B having a high output. Now, when the input signal goes low, the output of the inverter U1F is now high. Hence, across the capacitor circuit (R4 and C2; R4 was measured to be $11.00 \pm 0.14 k\Omega$ (by the DMM manual, uncertainty is 1.0 percent of the value plus 3, and C2 was measured at $0.963 nF \pm 70$ (once again according to plus or minus 5 percent plus 20, by the manual)), we suddenly have a potential difference, so current will begin to flow through the resistor, and begin charging the capacitor. Remembering that the capacitor cannot instantaneously (discontinuously) change its potential, there is a small delay before it charges to a certain threshold (we have just measured this voltage to be 2.98 V, with uncertainty 0.01 V), where the voltage at the input of U1B is still below its threshold for inverting high $\rightarrow$ low. This is where the delay comes from. As more time passes, current continues to flow through the resistor so that more charge, and hence potential difference, builds up on the capacitor (equipotential with the input to U1B), at which point the high $\rightarrow$ low threshold is reached. Qualitatively, this delay can be calculated by finding the time it takes for the capacitor

to charge from 0 volts to 2.98 volts. Seeing that the time constant of the circuit is $11k\Omega \cdot 0.963nF = 10.59\mu s$, we plug in $2.98 = 5(1 - e^{-\frac{t}{\tau}})$, giving us a theoretical delay of $9.6\mu s$. What we see in figure 8 is a delay of $10.56\mu s$, which is relatively close to our predicted value, and thus passes the sanity check.

```
In [10]: InlineImage('risingDelayMeasure.png', 8, 8, 'Fig. 8: Rising Trigger Delay Me
```

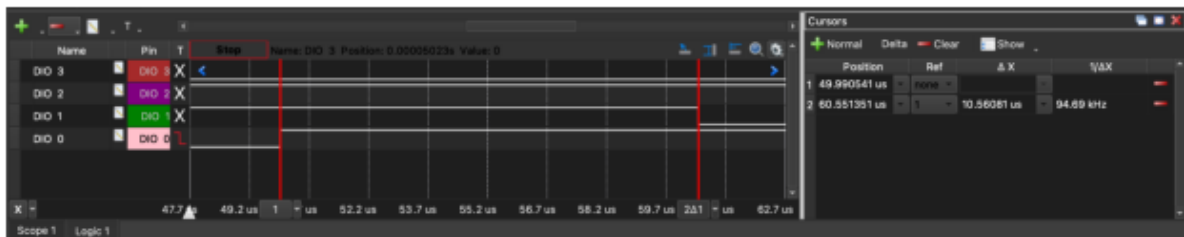


Fig. 8: Rising Trigger Delay Measurement

Now, when the input signal converts from low to high, the output of U1F is high, so that the capacitor is already fully charged (recall that the frequency of our input is very low, allowing plenty of time for full charging). Now that voltage at the capacitor is at a full voltage, but at the output of U1F it is low, the capacitor discharges through the resistor towards the output of U1F. Once the capacitor has discharged below 1.04 V (the measured hysteresis low to high voltage from a previous lab), the output of U1B transitions from low to high. The delay introduced here can be quantified as the time it takes the capacitor to discharge from 5 to 1.04 volts. With the same time constant as earlier, we calculate $1.67 = 5e^{-\frac{t}{\tau}}$, giving us a predicted delay of $t = 11.62\mu s$. Experimentally, we see a delay of $11.3\mu s$ here (see figure 9), which is also close to our theoretical value.

```
In [11]: InlineImage('resetDelayMeasure.png', 8, 6, 'Fig. 9: Reset Delay Measurement'
```



Fig. 9: Reset Delay Measurement

To see how the reset works, understand that when the output of U1B (which is the squared up, delayed input signal) is high for a long time, the capacitor is fully charged, so that the input to U1C is low, resulting in a low reset signal. When the input transitions to low, the capacitor has to maintain the same voltage across it, so the side connected to the resistor must change instantaneously to -5 V, so that current flows from ground to the negative plate of the capacitor. This whole time, the input to U1C is still low, so that the output for RESET is low. Now, when the output of U1B changes

from low to high, the capacitor cannot instantaneously change voltage, so the voltage at the input to U1C is high, which is what we see at the beginning of the pulse, only when the delayed signal transitions from low to high. The reason this pulse is short however, is that the capacitor begins to charge, so that the voltage at the input to U1C decreases. Once it drops below the hysteresis low to high threshold of 1.67 V (corresponding to the capacitor charging from 0 to 3.33V), the signal is inverted again, so that the reset is low. The time that the reset is high can be calculated with the time it takes the capacitor C3 (measured experimentally to be $25 \pm 21\text{pF}$ (large uncertainty from manual, which states an accuracy of 5% plus or minus 20, so that 46 pF is within the realm of possible capacitances (important to note for a later note)) to charge from 0V to 3.33V. The time constant (with R5 measured at $10.87 \pm 0.01\text{k}\Omega$ is $\tau = 25\text{pF} \cdot 10.87\text{k}\Omega = 0.272\mu\text{s}$. Inputting the numbers as earlier with $3.33 = 5(1 - e^{-\frac{t}{\tau}})$, we get $t = 0.298\mu\text{s}$. This is approximately half of our measured value (see figure 10), which was $0.578 \pm 0.08\mu\text{s}$ (uncertainty measured by taking the range of possible measurements and dividing by four). Interestingly, when I perform this calculation with the expected capacitance of 47pF, I get a value of 0.56 microseconds, which is extremely close to our experimental value. Considering how great the uncertainty is and that 46 pF is at the edge of our range of possible capacitances based on the experimental measurement, this is not disconcerting.

TS As I was initially measuring C3, which should be 47 pF, I was measuring it with the prongs in my hand. What I ended up seeing was a wavering measurement that never landed on any capacitance, with the capacitance 10 times larger than expected. I thought this was peculiar, so I tried measuring it differently; with the capacitor resting on the paper, I rested the two prongs of the DMM on the capacitor, and now I was seeing a more expected result; while I measured $25 \pm 21\text{pF}$, we are within the correct magnitude of our expected capacitance.

In [12]: `InlineImage('resetPulseMeasurement.png', 10, 7, 'Fig. 10: Reset Pulse Length`

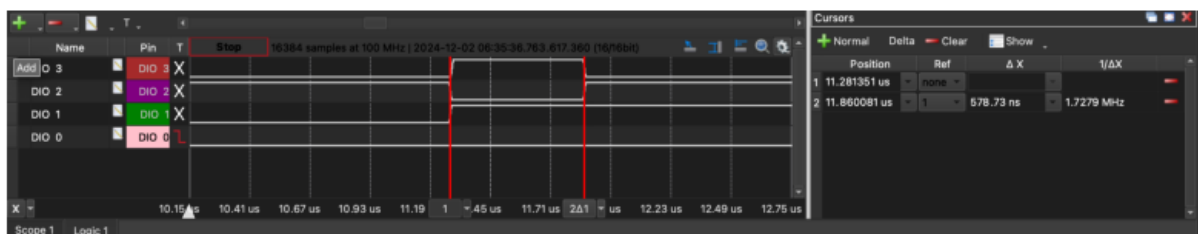


Fig. 10: Reset Pulse Length Measurement

4.2

I began by just trying to get the 256 bit counter to work. I connected the wiring as shown in figure 3 of the instructions page. I opted for staying on the same side of the board for this week, not committing to more wiring then needed to test the functionality of what I have so far. Shown below in figures 11-13 are the datasheet

figures for the counter, the 74HC393. Since it contains two four-bit counters, we can do the whole 8-bit counter on one IC. See figure 14 for the circuit diagram, and figure 15 for the breadboard circuit.

```
In [13]: InlineImage('74HC393pinout.png', 7, 7, 'Fig. 11: 74HC393 pinout', 14)
```

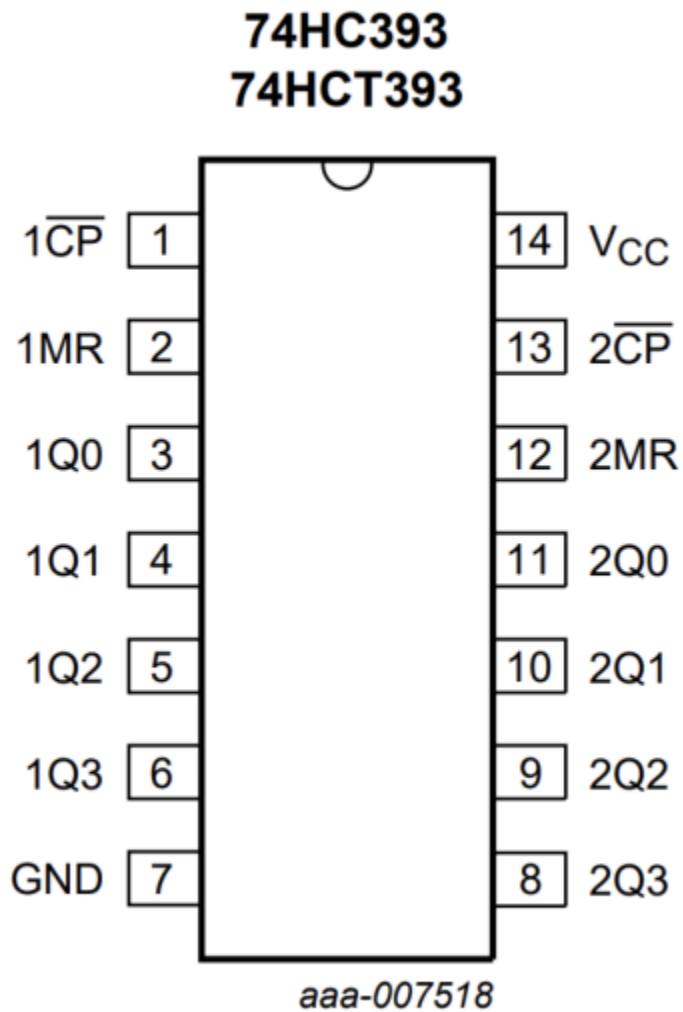


Fig. 11: 74HC393 pinout

```
In [14]: InlineImage('74HC393funcDiag.png', 6, 6, 'Fig. 12: 74HC393 Functional Diagram')
```

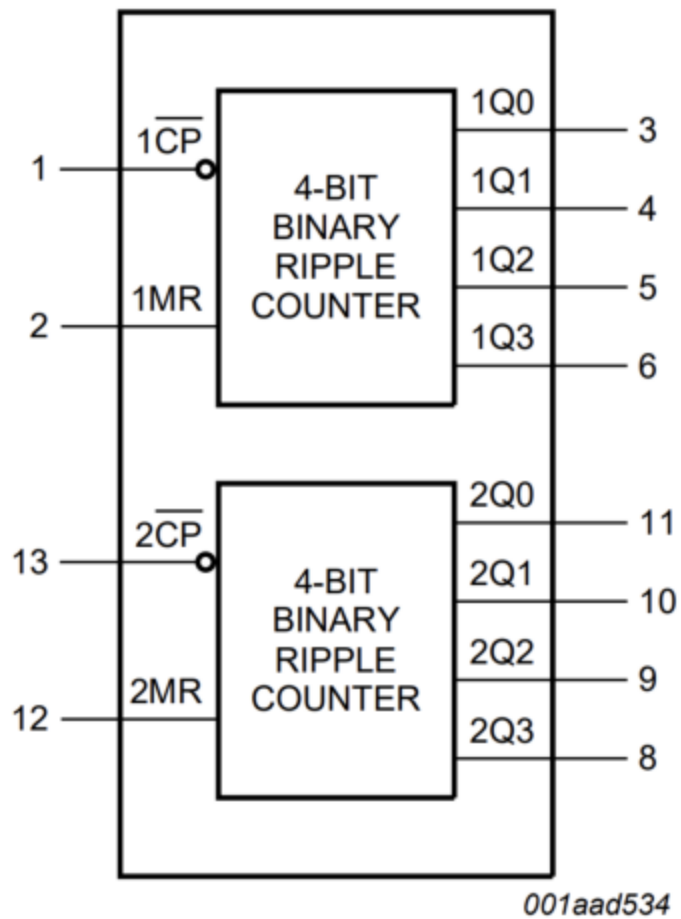


Fig. 12: 74HC393 Functional Diagram

```
In [15]: InlineImage('74HC393logicSymb.png', 6, 6, 'Fig. 13: 74HC393 logic symbols',
```

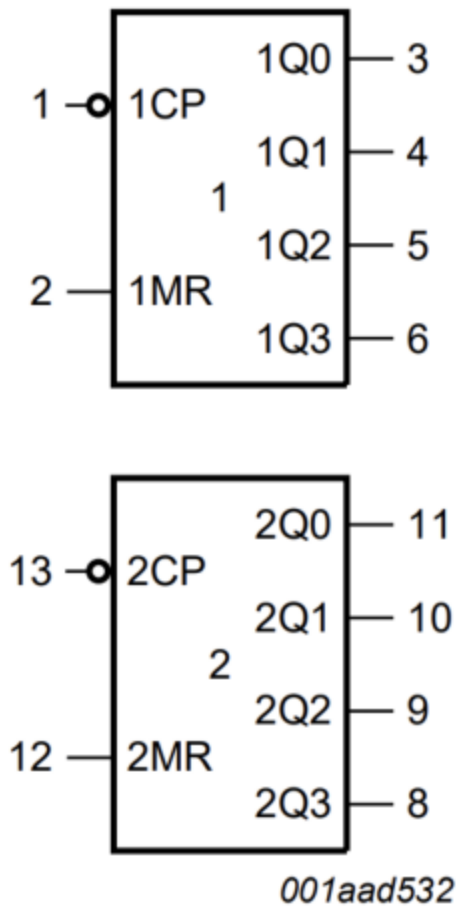


Fig. 13: 74HC393 logic symbols

```
In [16]: InlineImage('counterCircDiag (1).jpeg', 6, 6, 'Fig. 14: 16-bit counter diagr
```

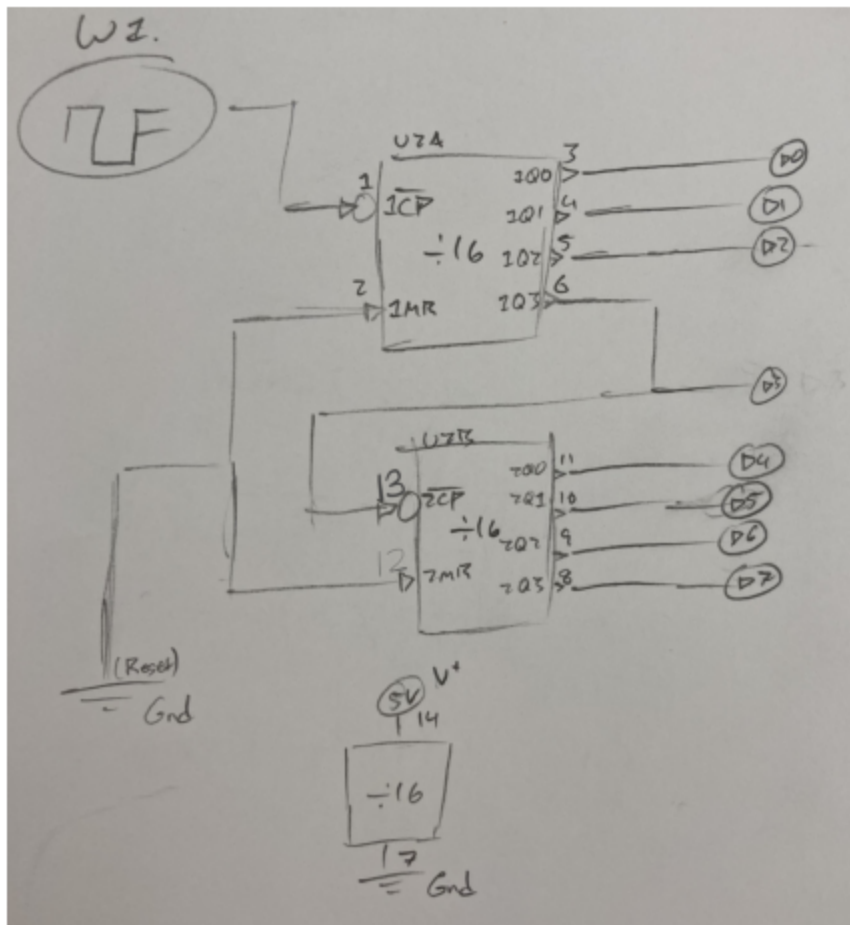



Fig. 14: 16-bit counter diagram

In [17]: `InlineImage('255CounterTest.jpeg', 10, 10, 'Fig. 15: 255 counter breadboard`

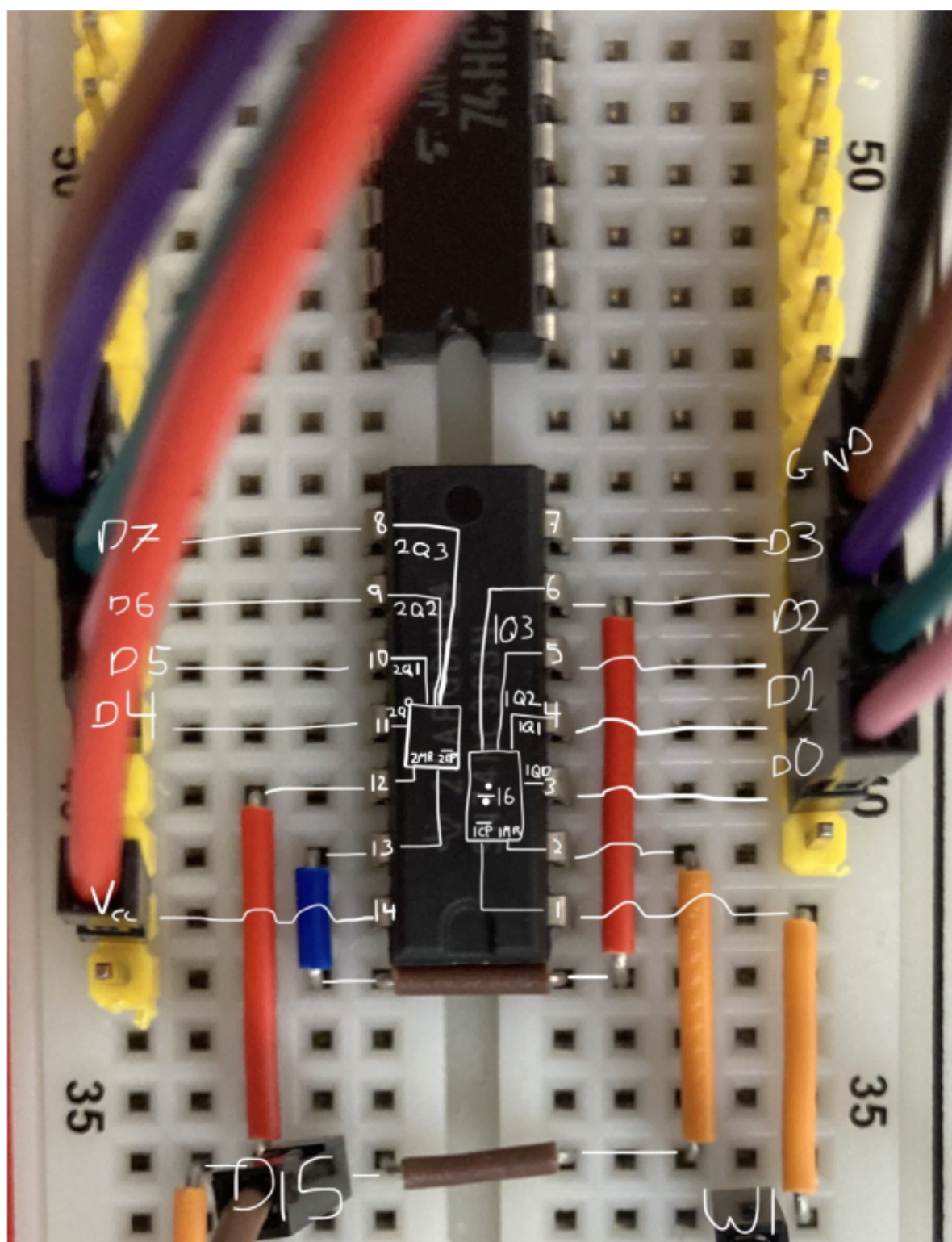


Fig. 15: 255 counter breadboard circuit

As we explored in an earlier lab, the counter works by counting four-bits with the first counter (1Q0-1Q3), and feeding the output of the most significant bit (1Q3) into the input for the next counter (2CP), which only registers a count on the rising edge of the most significant bit. In this way we have 16 states for the second counter, each one occurring once every 16 states for the first counter, giving us 256 total states. Using the bus on the logic analyzer, we can convert this into a decimal output to see that our

circuit works. See figure 16 for the output. Note that we had to ground the reset pins while we weren't using them, because when they float they constantly the counter to 0, which we naturally don't want in a counter. See below in figure 16 that the counter counts to 255 and then cycles back to 0. We will discuss this soon.

```
In [18]: InlineImage('countTo255.png', 6, 6, 'Fig. 16: 8-bit counter logic', 14)
```



Fig. 16: 8-bit counter logic

In order to get the reset working, I added a patterns with DIO 15, and just set it to constant, setting to high, and then low, and then saw the behaviour in figure 17, whereby the count is reset to 0, and then starts counting again when the reset is set low again.

```
In [19]: InlineImage('resetFuncCounter.png', 7, 7, 'Fig. 17: Reset Functionality Logi
```

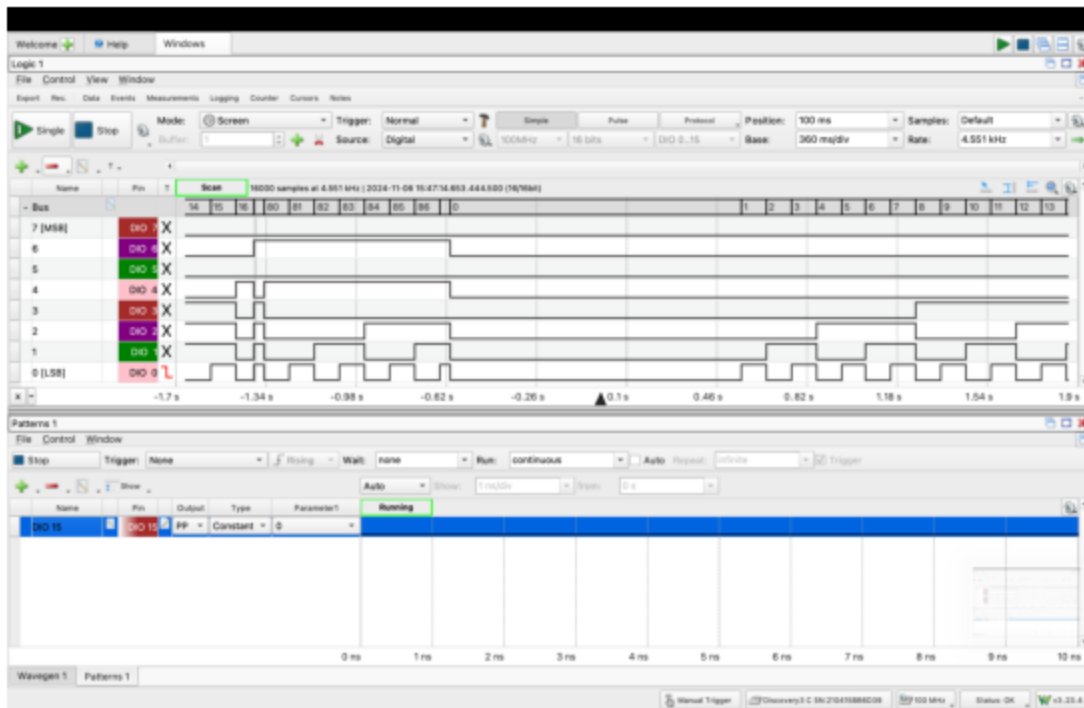


Fig. 17: Reset Functionality Logic

Moving on, I now tried to use the push button to reset the counter at the push of a button. I opened a static IO window, and set DIO 15's settings to be such that released is 0 (ground, or low), and pressed is high. In this way, when I pressed the button, I would encounter a reset in the counter, as desired (see figure 18).

```
In [20]: InlineImage('resetWithButton.png', 8, 8, 'Fig. 18: Reset push button counter')
```



Fig. 18: Reset push button counter functionality

My counter counts all the way up to 255, before resetting to 0 and repeating the counting process. This is built into the nature of how counters work; when they reach the end, they go back to 0; in the case of the four bit counter, our being able to count to 255 relied on the first counter counting 0 to 15, before clocking into the next counter, and then counting once again from 0 to 15; this cyclic behaviour of the counter is crucial, and the exact same phenomenon results in the 8-bit counter resetting to 0; when the second clock is at 15 (all bits set to 1), and another clock pulse is input, similarly to with the first counter, all the bits reset to 0 in its cyclic nature, so we reset to 0 (as the first clock resets to 0 as well).

Integrating the counter with the D-latch

Shown below in figures 19-21 are the datasheet figures for the D-latch IC we are going to use to latch the bits from the counter at any given moment.

```
In [21]: InlineImage('74HC273pinout.png', 7, 7, 'Fig. 19: 74HC273 pinout diagram', 14
```

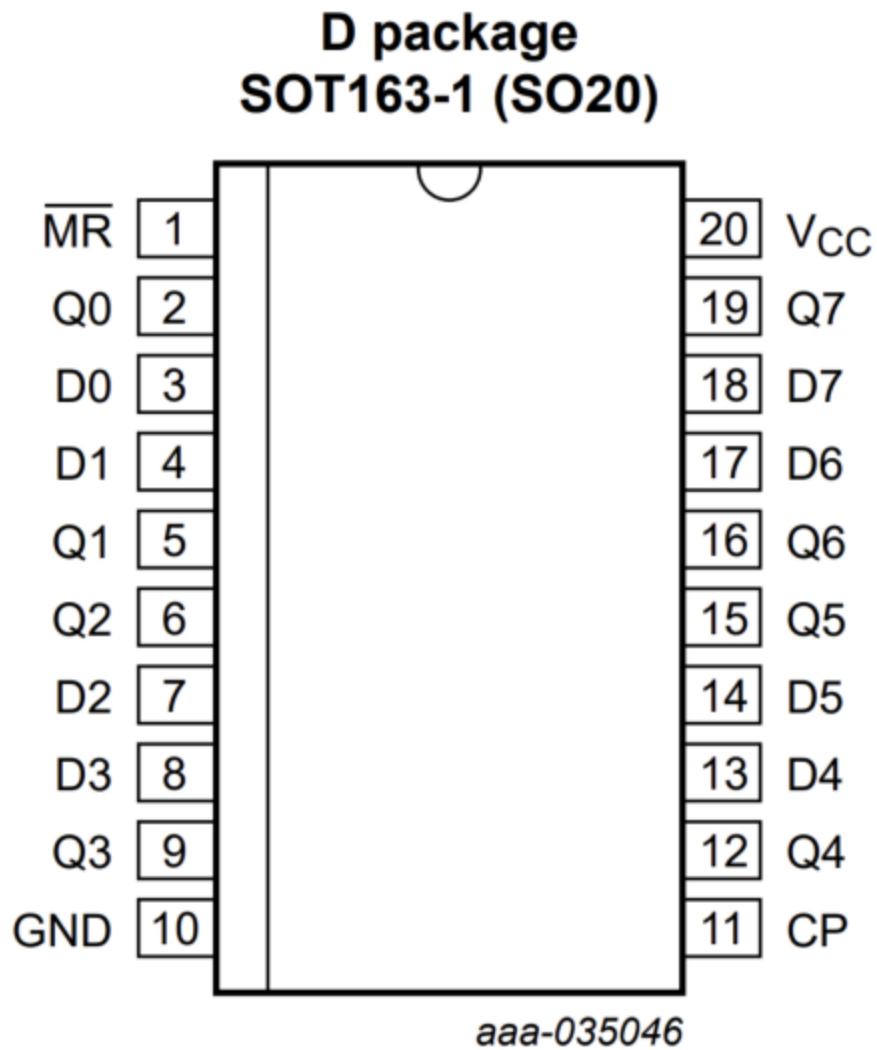


Fig. 19: 74HC273 pinout diagram

```
In [22]: InlineImage('74HC273funcDiag.png', 7, 7, 'Fig. 20: 74HC273 Functional Diagram')
```

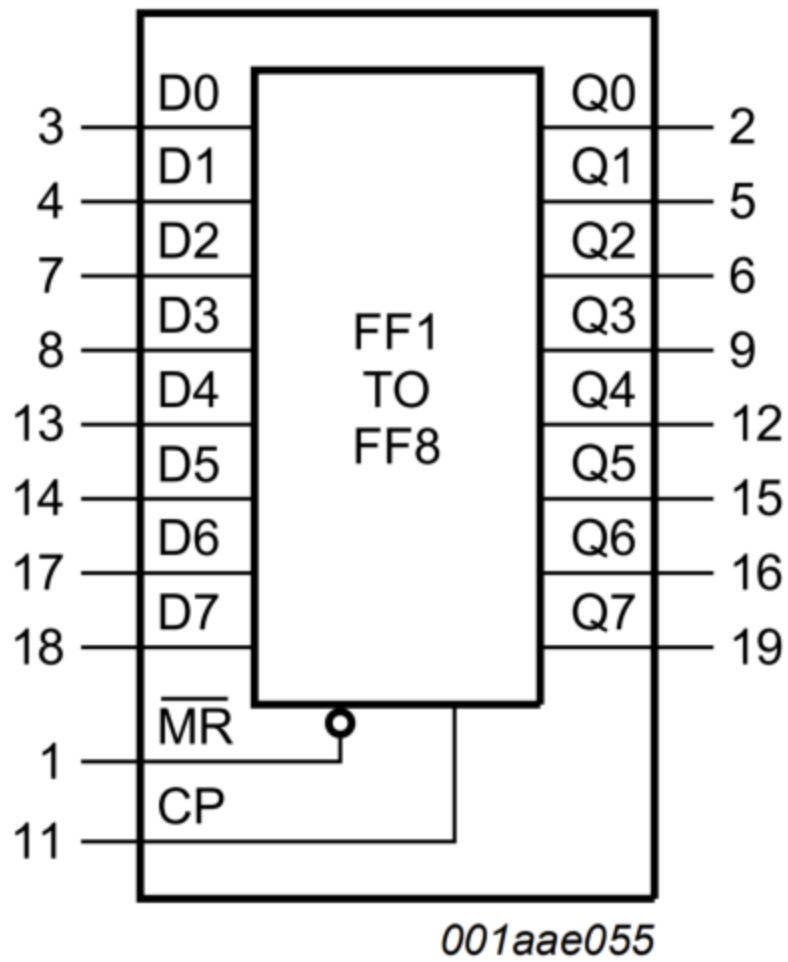


Fig. 20: 74HC273 Functional Diagram

In [23]: `InlineImage('74HC273logicDiag.png', 6, 6, 'Fig. 21: 74HC273 Logic Diagram',`

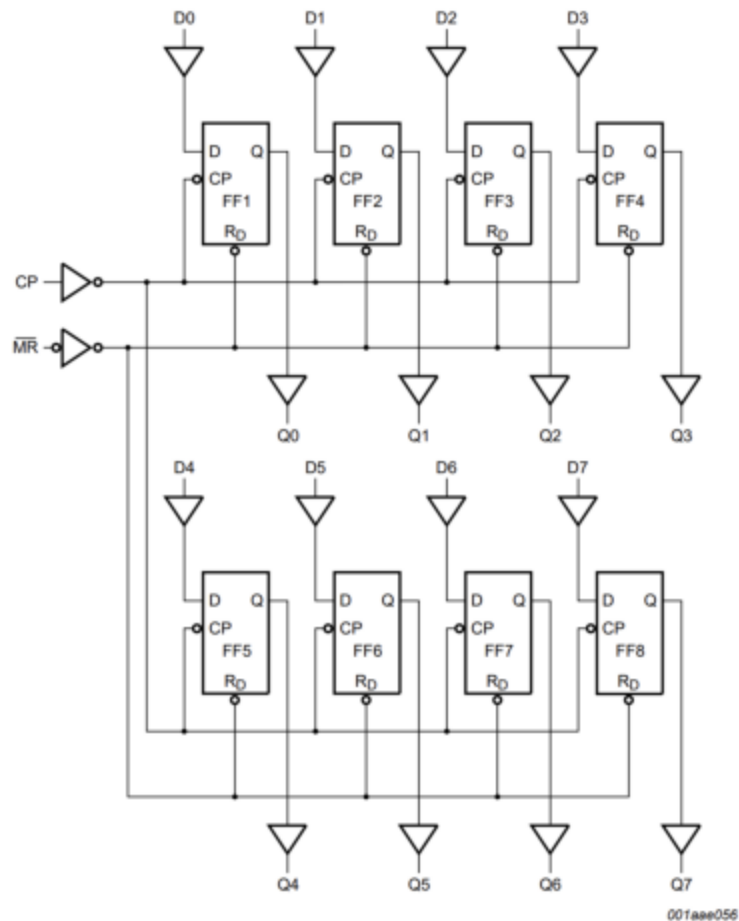
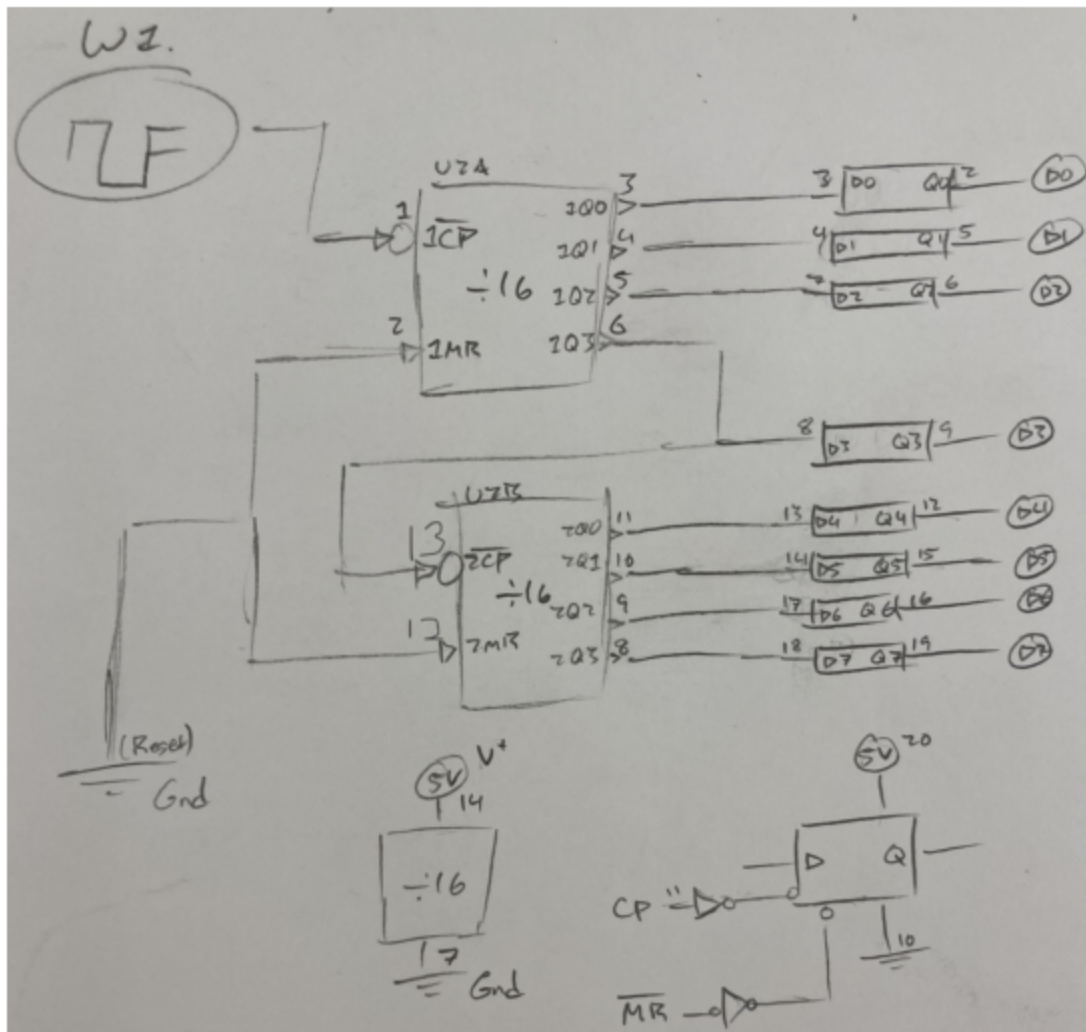


Fig. 21: 74HC273 Logic Diagram

I wired up all of the outputs of the counter to the inputs of the D-latch, always with the datasheet pinout right in front of me, to make sure no mistakes were made. See Figure 22 and 23 for the testing setup diagram, and breadboard circuit, respectively. I then connected the clock pulse to D15, which I controlled via push button so I could manually latch the data. Since this testing in isolation of the D-latch clock is just a test, I made some temporary wiring solutions to connect everything to Vcc (I needed three things to connect to high: MR (master reset), and Vcc on the clock and the D-latch. I will integrate this much more smoothly in the final circuit design). After confirming several times over that I was confident in my wiring, I ran my patterns and supply, with the same bus settings as in the counter.

In [24]: `InlineImage('dlatchDiag.jpeg', 7, 7, 'Fig. 22: D-latch Circuit Diagram', 12)`



```
In [25]: InlineImage('dlatchTestSetup.jpeg', 11, 11, 'Fig. 23: D-latch testing breadb
```

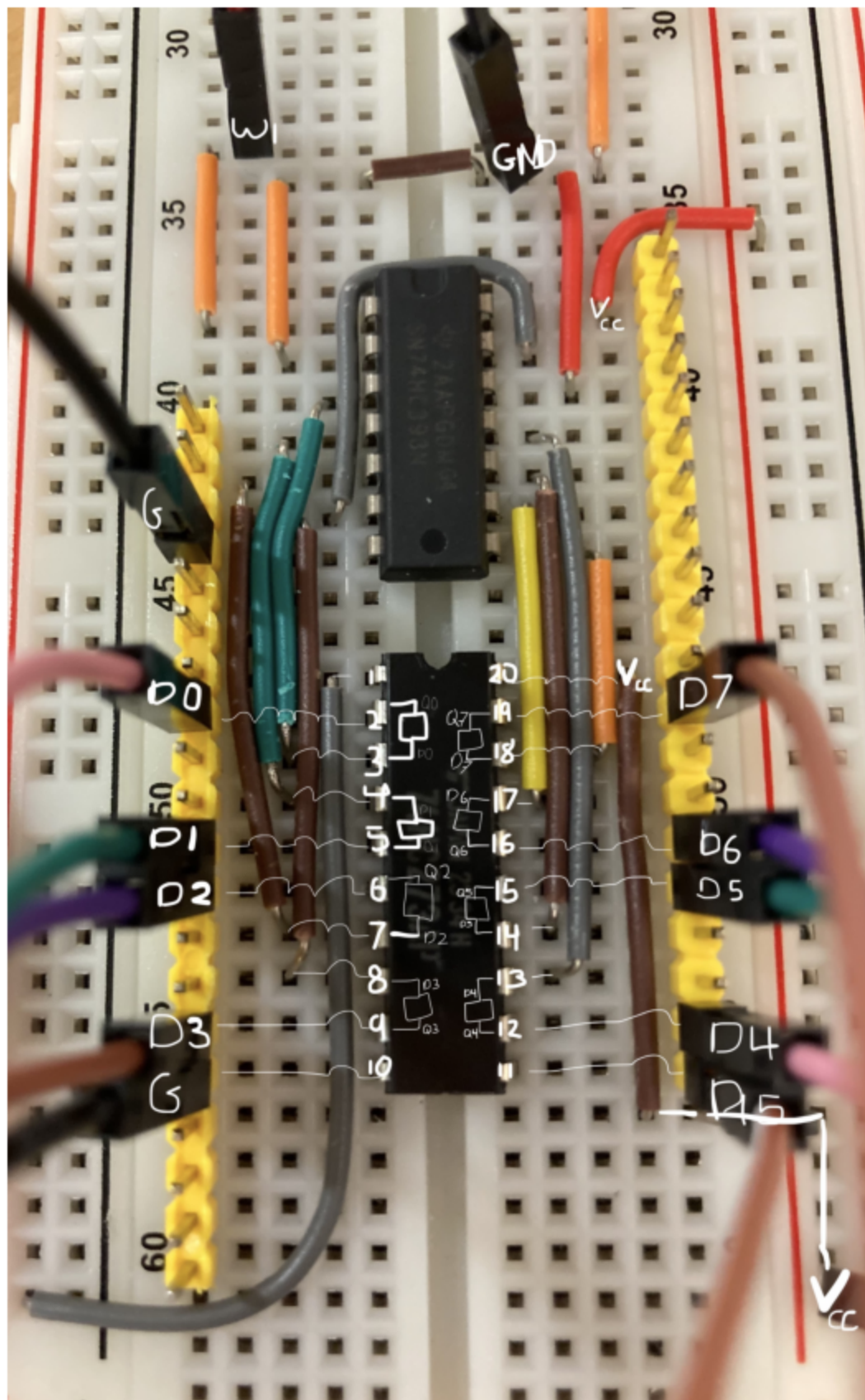



Fig. 23: D-latch testing breadboard setup

To my surprise this worked as expected, right away. To operate it (see figure 24), I pressed the push button each time I wanted to latch the output bits of the counter into

the D-latch latched bits; and I knew it was working because I saw that when I pressed the button quickly and consecutively, the output latched on the D-latch matched what the counter should be counting, and the reset from 255 to 0 also worked as expected (see figure 24).

```
In [26]: InlineImage('D-latchWorking.png', 7, 7, 'Fig. 24: D-latch working', 12)
```

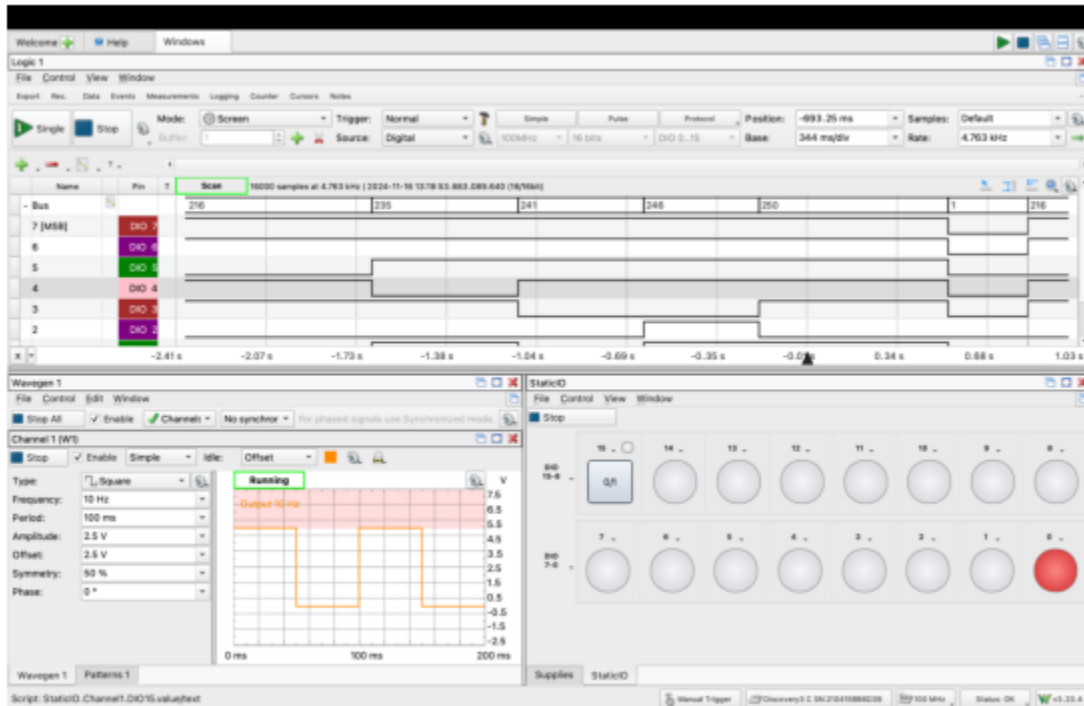


Fig. 24: D-latch working

TS

Going back later in order to test the reset functionality with the push button in another session, I had already connected the wiring from the counter to the D-latch, and I tried remaking the circuit as shown in figure 11, but was surprised to see that the counter was acting sporadically; generally it was counting sequentially but there were often outliers that had no rhyme or rhythm. Since generally it was counting correctly, I knew my wiring and bus settings were correct, but something else must be going on. Sure enough, when I tried disconnecting the D-latch, the problem resolved itself; I was now getting nominal counting behaviour. There may have been cross-talk between different outputs in the D-latch which led to my odd behaviour.

The next step is to wire up the output of the D-latch into the DAC (see figure 25 for datasheet pinout). It was at this point that I realized I had way too much empty space between the delay circuit at the counter circuit and d-latch. Thankfully I'm not too far in, so I was able to relatively quickly move everything together to be much more compact, which also gave me the opportunity to be more compact with my wiring. After doing this, once again while looking very carefully at the D-latch datasheet, I

connected the 8 output bits from the D-latch onto the DAC. This allowed me to much more compactly measure the output of the D-latch, with 8 DIO's directly adjacent to each other. I measured the output of the DAC using the scope, with the negative scope connected to ground.

Since I wanted to see that the DAC was always working, and I had already tested my delay circuit in isolation, I decided to integrate it together. To start, I placed my waveform as in task 4.1 into the delay circuit, and connected the reset output to ground (since I want to use the input waveform as the clock, I can't also have the reset pin having the same frequency as the clock, otherwise the counter would be stuck at 1 (if the delay works properly), so I grounded the reset). I then connected my latch to output to the CLK on the d-latch (pin 11), and also used the same latch signal to clock the counter. With these changes made, the circuit is looking a lot cleaner; see figure 26 for the D-latch and DAC combination diagram, and 27 for the breadboard circuit. (I only annotate the parts of this picture that are unique to it)

```
In [27]: InlineImage('LF103pinout.png', 7, 7, 'Fig. 25: 4610X-R2R-103LF pinout', 16)
```

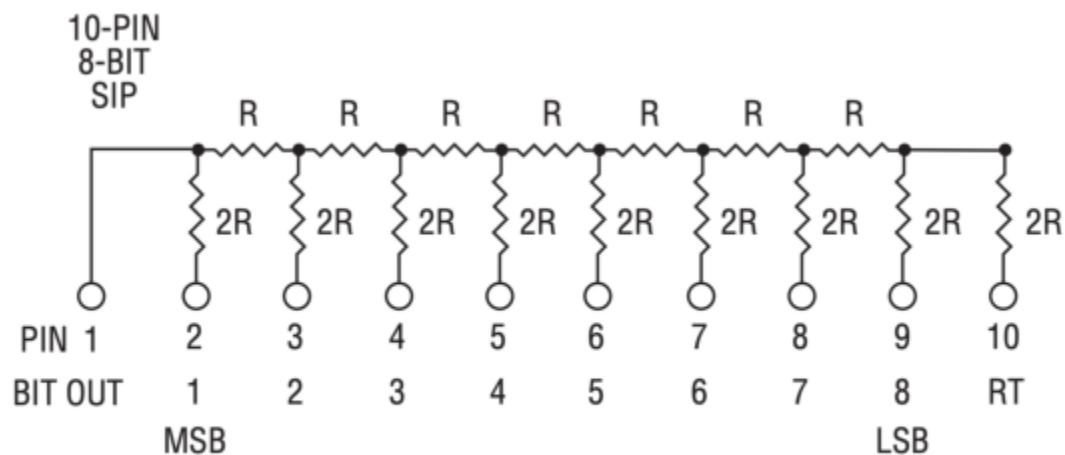
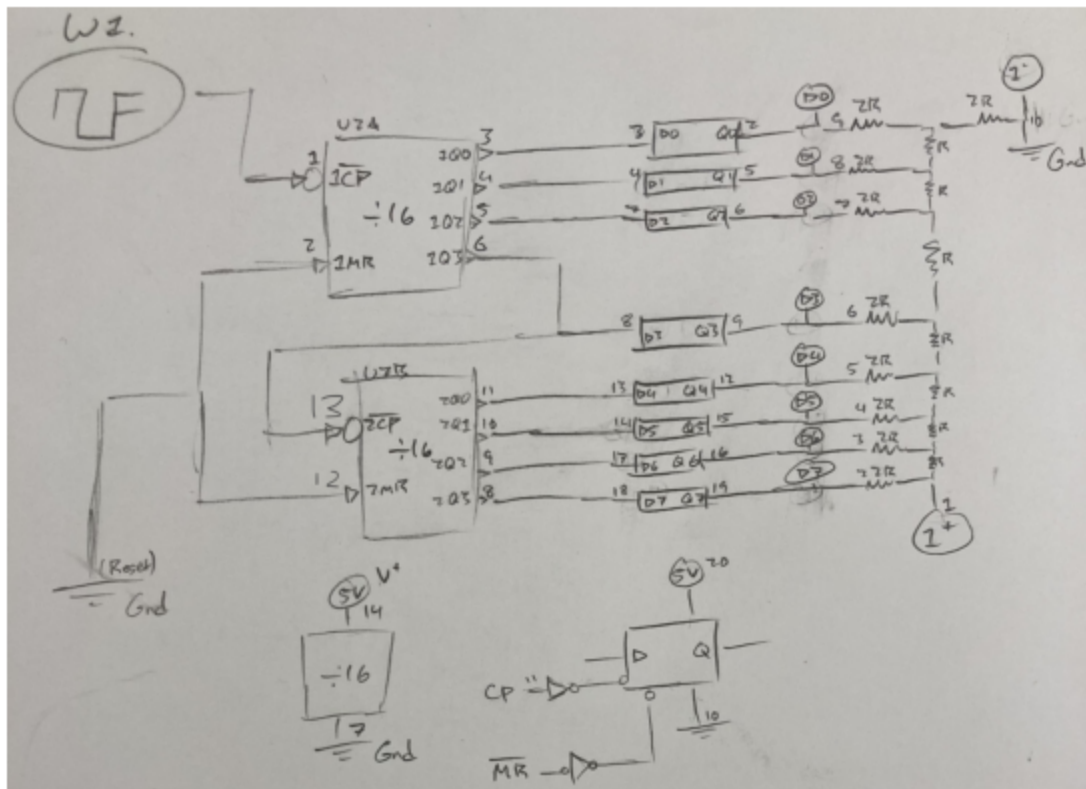


Fig. 25: 4610X-R2R-103LF pinout

```
In [28]: InlineImage('dacDiag.jpeg', 7, 7, 'Fig. 26: DAC integrated with counter and
```



```
In [29]: InlineImage('dacPic.jpeg', 10, 10, 'Fig. 27: DAC breadboard picture', 10)
```

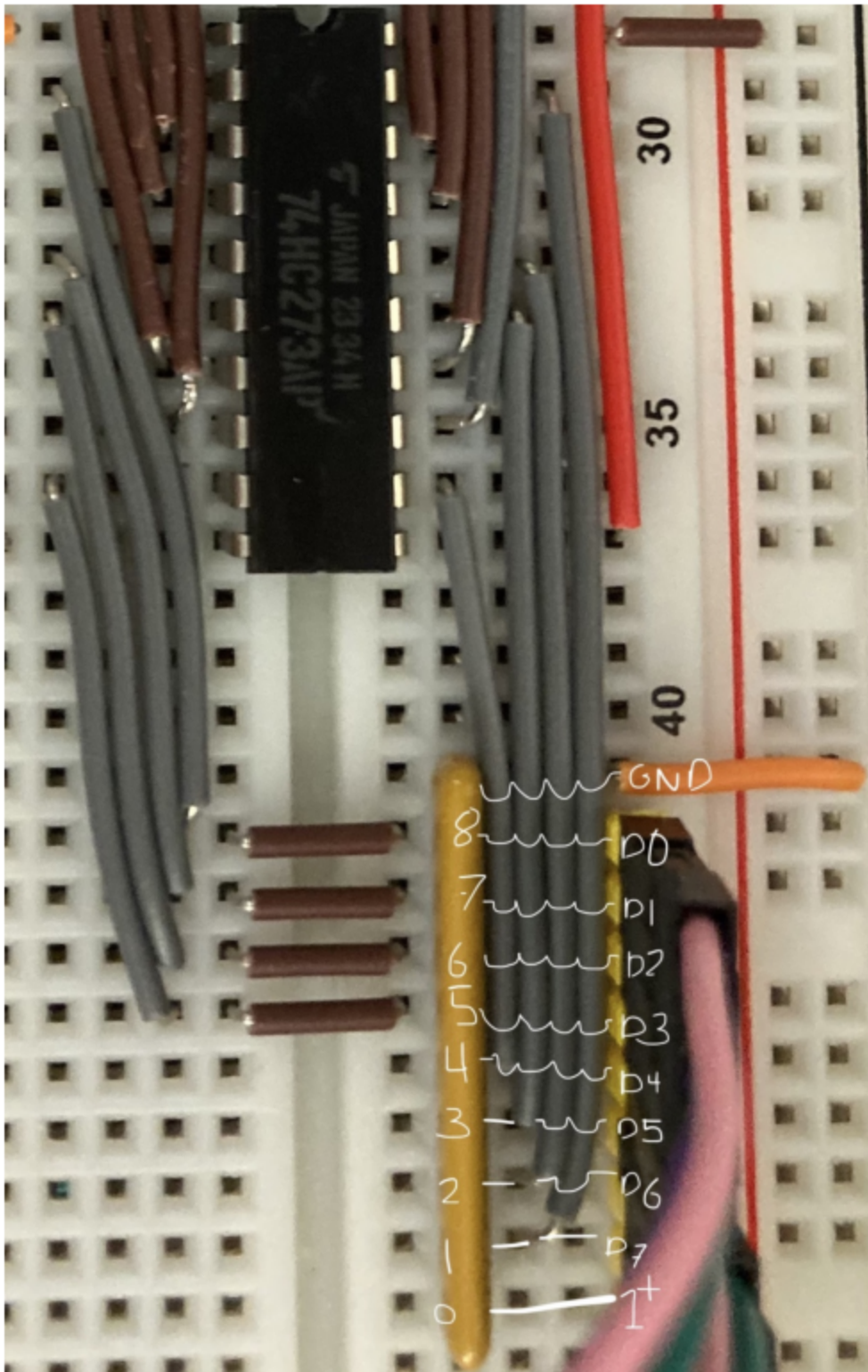



Fig. 27: DAC breadboard picture

We can see that the DAC works as expected in figure 28; based on the scope trace, the voltage at the output of the DAC is linearly proportional to the value of the bus from the logic analyzer.

```
In [30]: InlineImage('DACworking.png', 7, 7, 'Fig. 28: working DAC scope trace', 12)
```

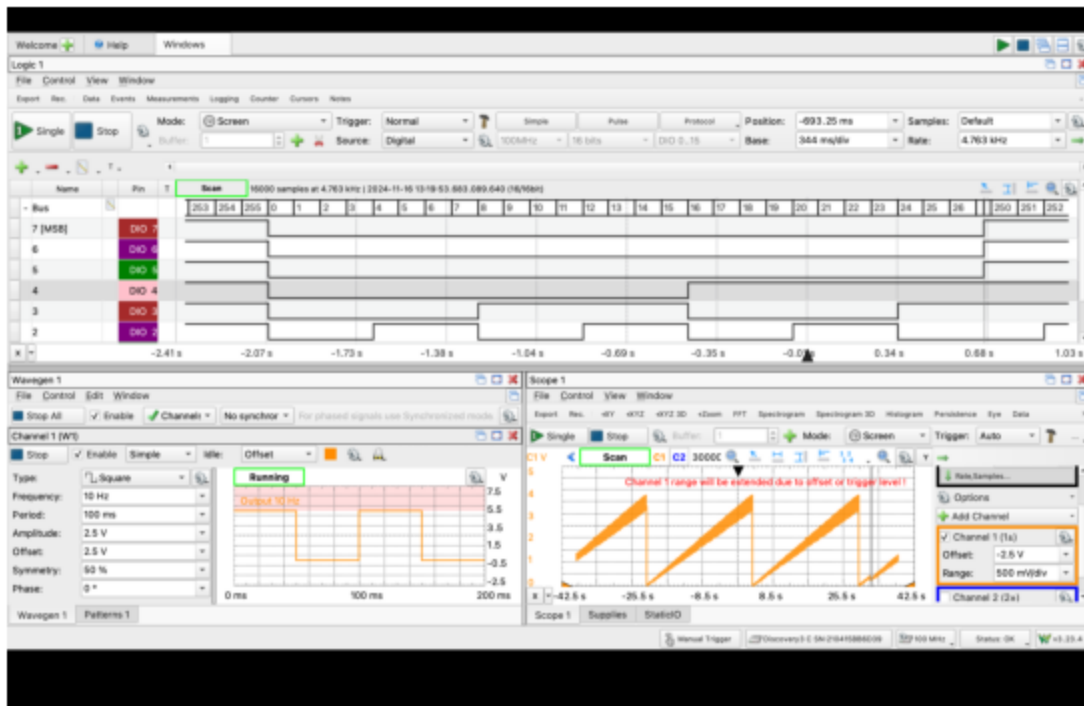


Fig. 28: working DAC scope trace

Buffer

In [31]: `InlineImage('LF411pinout.png', 7, 7, 'Fig. 29: LF411 Operational Amplifier P`

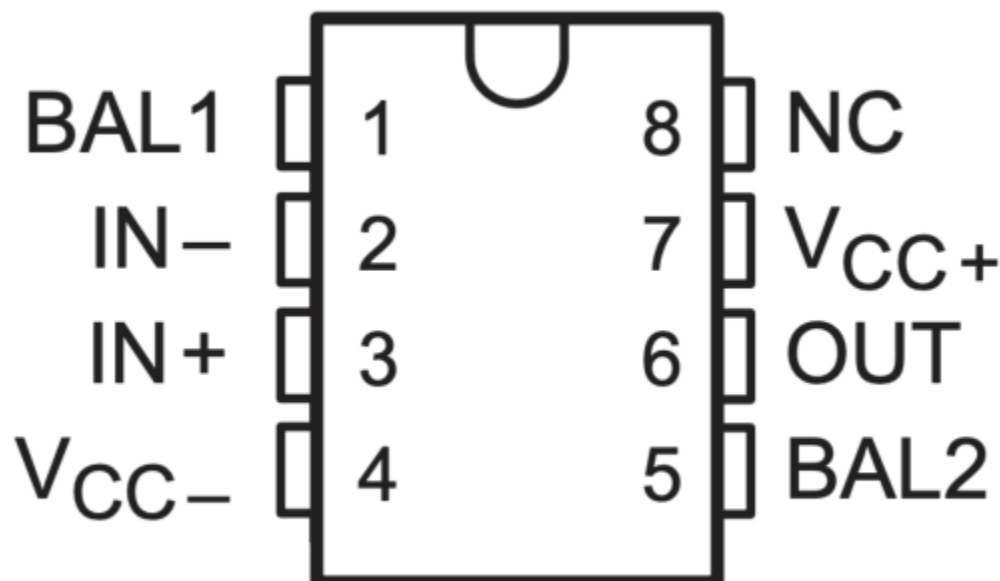


Fig. 29: LF411 Operational Amplifier Pinout

Lastly, I need to add the buffer to the output signal of the DAC. See figure 29 above for the pinout of the operational amplifier that we use in this circuit: the LF411.

TS

I started by connecting the output of the DAC to IN+, and the output pin to IN-. I connected Vcc+ to the voltage supply, and Vcc- to ground. What I saw when I ran the experiment the same as with just the DAC was an extremely noisy wave that did not resemble anything like the output of the DAC. After pondering a bit and looking through the pin diagram, I remembered from previous weeks that Vcc- was always equal to the negative of Vcc+. Realizing my mistake, I fixed it, and when I re-ran the scope with the same conditions as earlier, I saw figure 32, as expected. See figure 30 and 31 for the wiring of the op-amp, with the whole circuit included. We now see that the scope trace is more precise (albeit sometimes noisy) than it was without the buffer.

In [32]: `InlineImage('bufferDiag.jpeg', 6, 6, 'Fig. 30: Buffer with counter and d-lat`

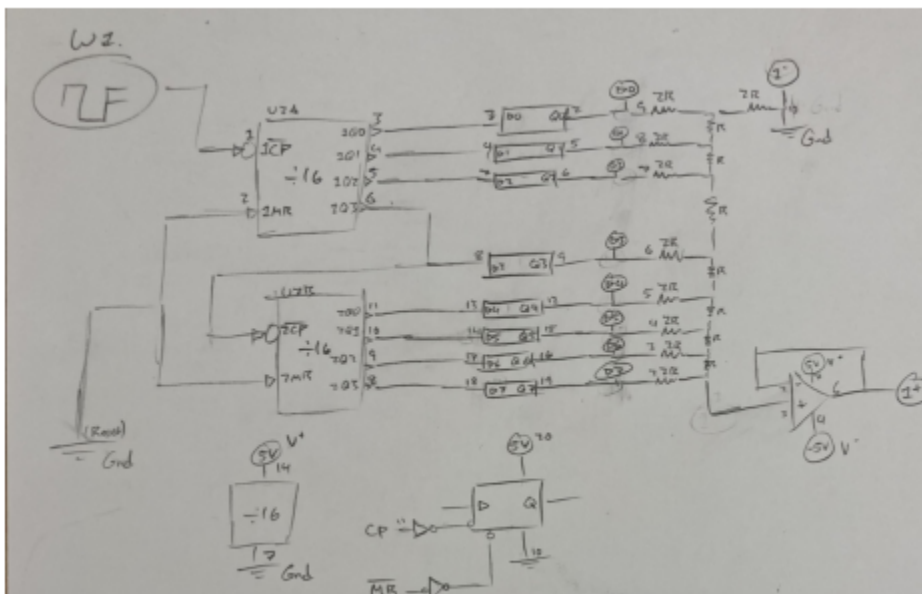


Fig. 30: Buffer with counter and d-latch diagram

In [33]: `InlineImage('bufferInclPic.jpeg', 8, 8, 'Fig. 31: Breadboard of buffer inclu`

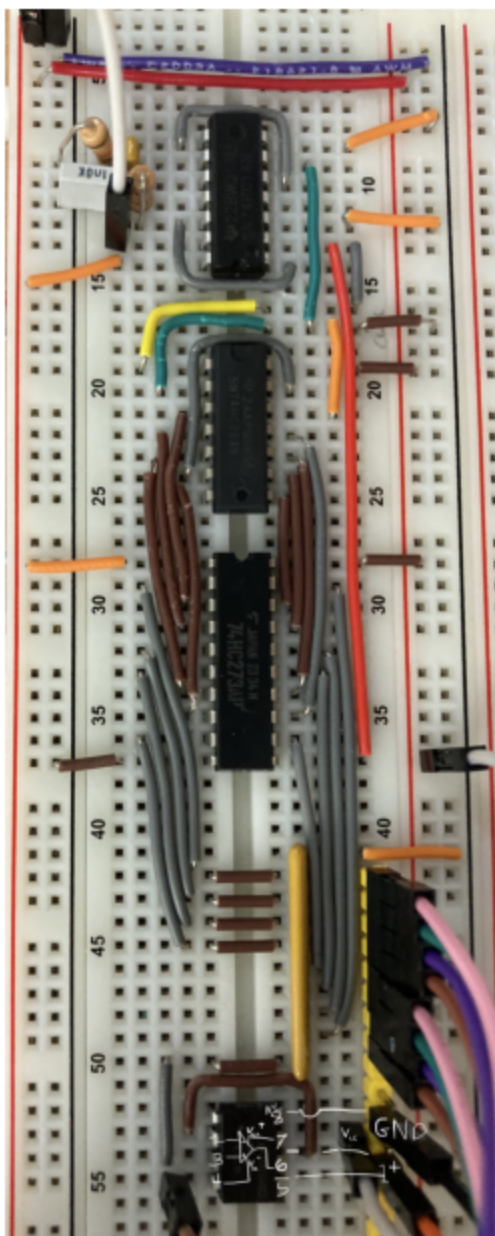


Fig. 31: Breadboard of buffer included

In figure 32 below, the waven is the input to the delay circuit which is configured as the input to the clock on the counter, with the latch signal from the delay circuit connected to pin 11 on the D-latch. The scope is being taken from the output of the latch (1+) to ground (1-).

```
In [34]: InlineImage('bufferWorking.png', 8, 8, 'Fig. 32: Working Buffer Scope Trace')
```

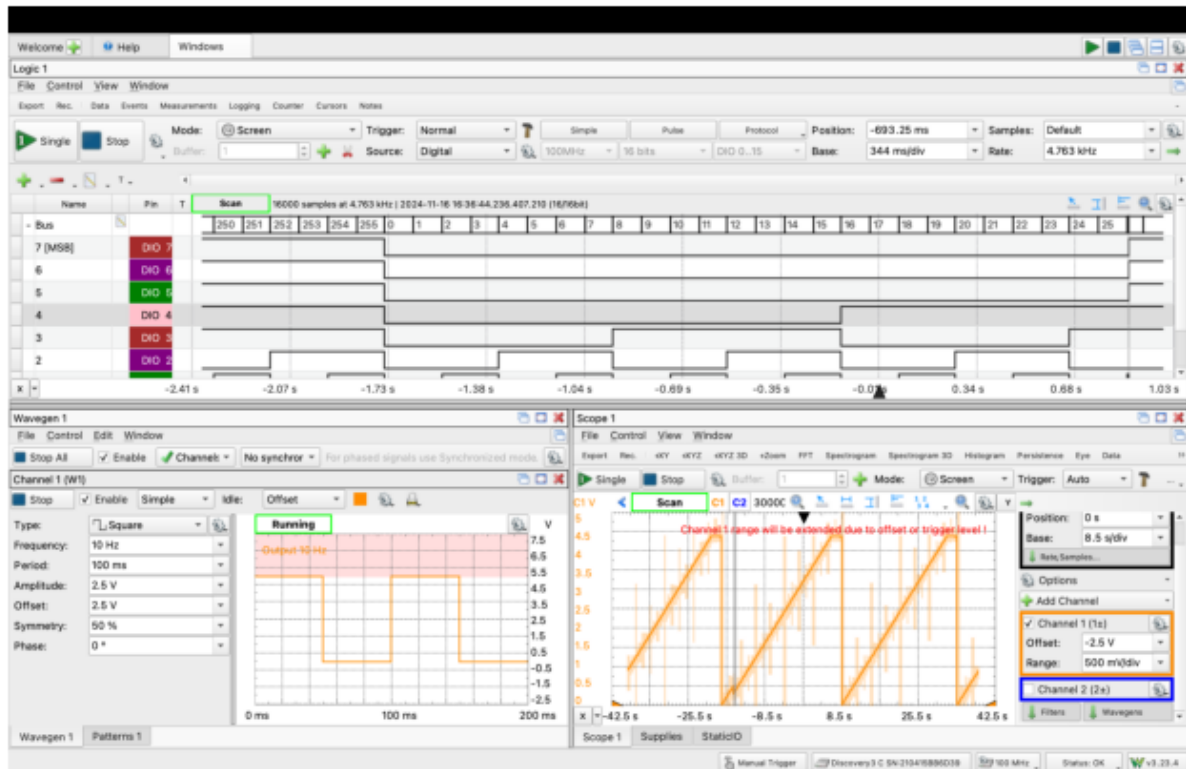



Fig. 32: Working Buffer Scope Trace

4.4 Error Amplification

We start by deriving an expression for the output of the error amplifier with respect to the input. First, let's examine the voltage at the positive terminal of the operational amplifier U5. The voltage supplied will be given by the voltage divider with the potentiometer; so $V^+ = 5\left(\frac{R_9}{10k\Omega + R_9}\right)$. In this way, we can adjust the speed of the motor with the potentiometer by adjusting the desired speed according to the resistance. Assuming an ideal op amp, we have $V_- = V^+$, so $V_- = \frac{5R_9}{10k\Omega + R_9}$. Now the input current will then be given by $I = V/R$, so that $I_{in} = \frac{V_{in} - V_+}{R_6}$. Since no current goes into the negative terminal of the operational amplifier (assuming ideal behaviour), we have that the current through $C1$ and R_7 is I_{in} . Now the voltage across the capacitor $C1$ is given by $V = Q/C$, or in terms of the current:

$V_c = \frac{1}{C} \int_{-\infty}^t i_{in}(t) dt$. The voltage across the resistor R_7 will be given by Ohm's law: $V_{R7} = i_{in} R_7$. Putting this together then and substituting our expression for $I_{in}(t)$,

$$V_{out} = V_+ - \frac{1}{R_6 \cdot C_1} \int_{-\infty}^t V_{in} - \frac{5R_9}{R_{10} + R_9} dt - R_7 \cdot \frac{V_{in} - \frac{5R_9}{R_9 + R_{10}}}{R_6}$$

Now substituting the numbers at hand for this application,

$$V_{out} = \frac{5R_9}{10k\Omega + R_9} - \int_{-\infty}^t V_{in} - \frac{5R_9}{R_9 + 10k\Omega} dt - 0.68 \left(V_{in} - \frac{5R_9}{10k\Omega + R_9} \right) \quad ($$

Testing my error control signal, it seems I am getting a time variant signal; when the input voltages goes very high, the output voltage goes low, and this behaviour centers around 2.5 V, which we expect to see. (Inserted later: see note above figure 37 about how the error amplifier was far too reactive).

In figures 33 and 34 are the circuit diagram and breadboard circuit for the error amplification circuit.

In [35]: `InlineImage('errorAmpDiag.jpeg', 7, 7, 'Fig. 33: Error Amplification Diagram`

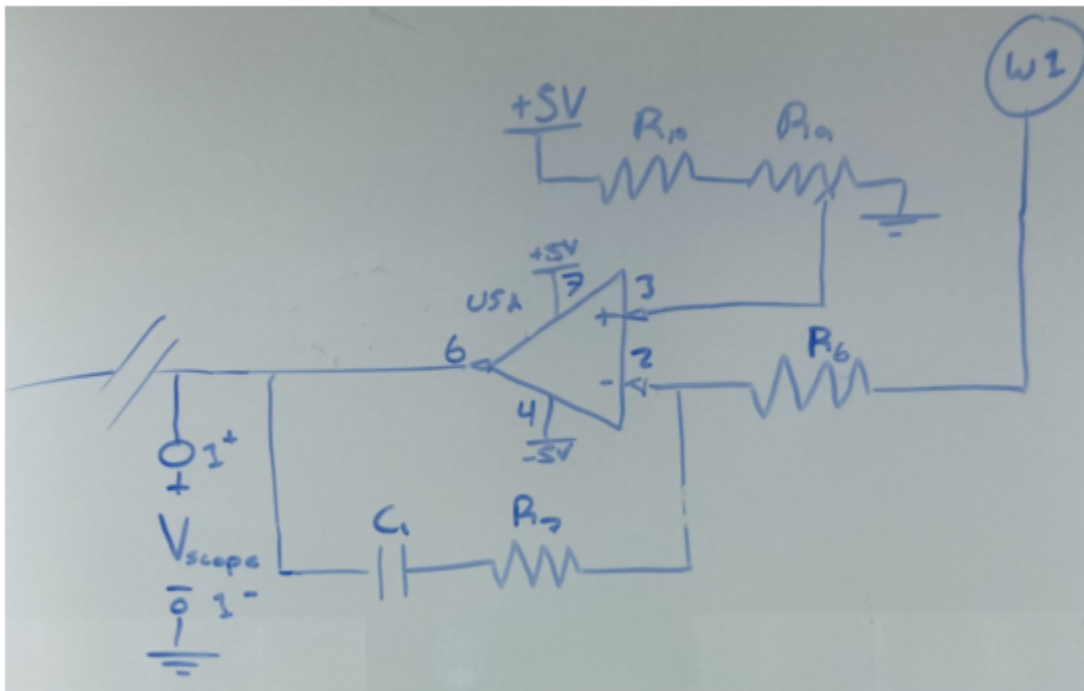


Fig. 33: Error Amplification Diagram

In [36]: `InlineImage('errorAmp.jpeg', 7, 7, 'Fig. 34: Error Amplification Circuit', 1`

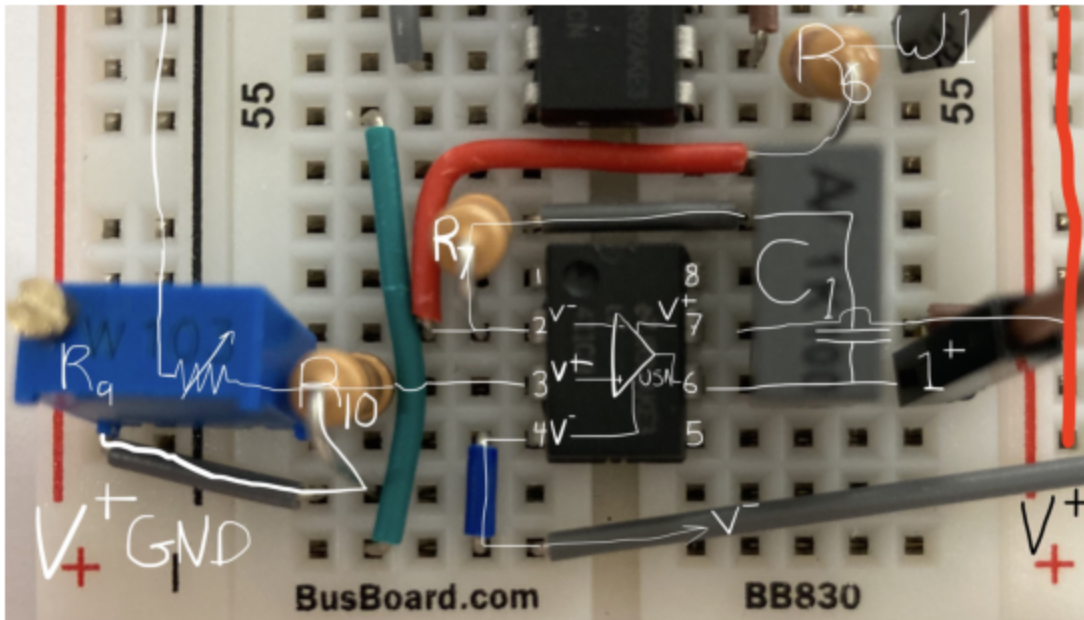


Fig. 34: Error Amplification Circuit

In [37]: `InlineImage('errorAmpScope.png', 6, 6, 'Fig. 35: Error Amp Scope', 14)`

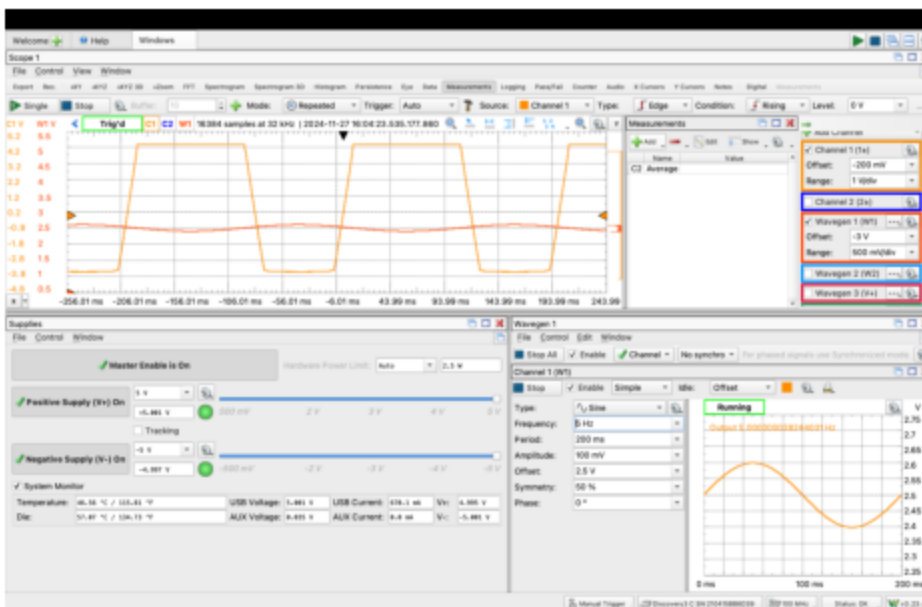


Fig. 35: Error Amp Scope

I more closely tested the amplification using a DC wavegen with small variations about the desired voltage (which I found was around 2.5 V). When I increased the input slightly above the expected, the error amplification started to decrease, and when I decreased the input slightly below the expected, the error amplification started to increase. This is hard to document, because it involves a changing output. As will be discussed, the reactivity seen in figure 35 is far too abrupt and not desirable for a control system; the mistake underlying this will be revealed in the troubleshooting section.

In [38]: `InlineImage('integratorErrorAmpScope.png', 7, 7, 'Fig. 36: Integrating Error`

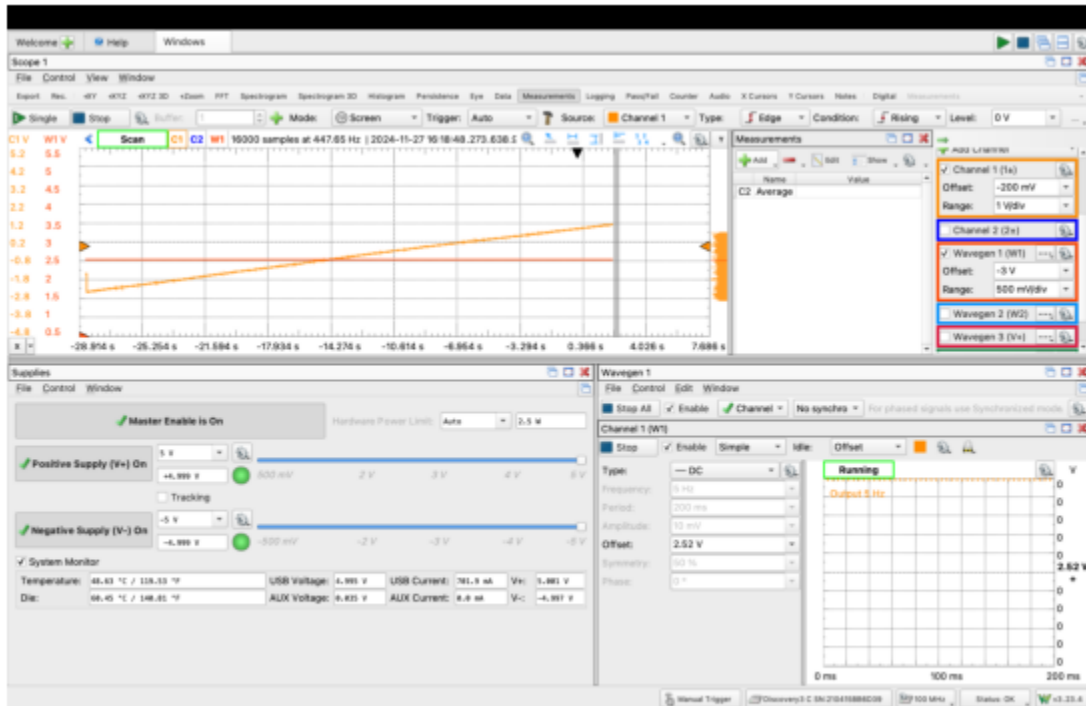


Fig. 36: Integrating Error Amp Scope

[Inserted later]. We will later see how there were issues with how abrupt the error correction is, which you can notice in figure 35, in which small deviations from the ideal lead to massive changes in the output. For a motor speed control system this is not ideal because we want the error correction to be smooth, making minute adjustments towards the correct speed, rather than over-correcting, which was the case.

TS

This occurred much later, after the whole circuit was built. What I noticed while running the circuit, which you can see in the scope reading in figure 35, is that the error correction isn't very smooth; the changes are quite abrupt, so it seems the error correction terms are given more precedence than they deserve. Luckily while going back later to measure resistors I found that I had accidentally placed a $4.7\text{ k}\Omega$ resistor as R_6 in place of the $1\text{ M}\Omega$ resistor that should have been there. After making this swap, with the resistor this time reading $0.992 \pm 0.001\text{ M}\Omega$, I closed the circuit again with the positive wire of the motor and found a perfectly smooth rpm! It is now working exactly as expected, and as it was running, I tried turning the potentiometer to change the ideal voltage, and accordingly, the motorspeed followed the set voltage determined by the voltage divider.

In light of knowing my error amplification was far too reactive earlier, it seemed right to me to show/explore the function of the error amplification with the less abrupt

reaction, as it makes for a much more enlightening analysis; I used the same testing setup as in figure 28, with of course the replaced resistor, and the scope measuring from the output at pin 6 to ground. Inputting a triangle signal with offset slightly above the set voltage V^+ (which I adjusted to be exactly 2.5 V using the potentiometer and scope), I was able to see both the proportional and integrator terms at work when viewing over a much larger time scale than the period of the input. See figure 37.

In [39]: `InlineImage('errorAmpScopeFixed.png', 8, 8, 'Fig. 37: Error Amplification Sc`

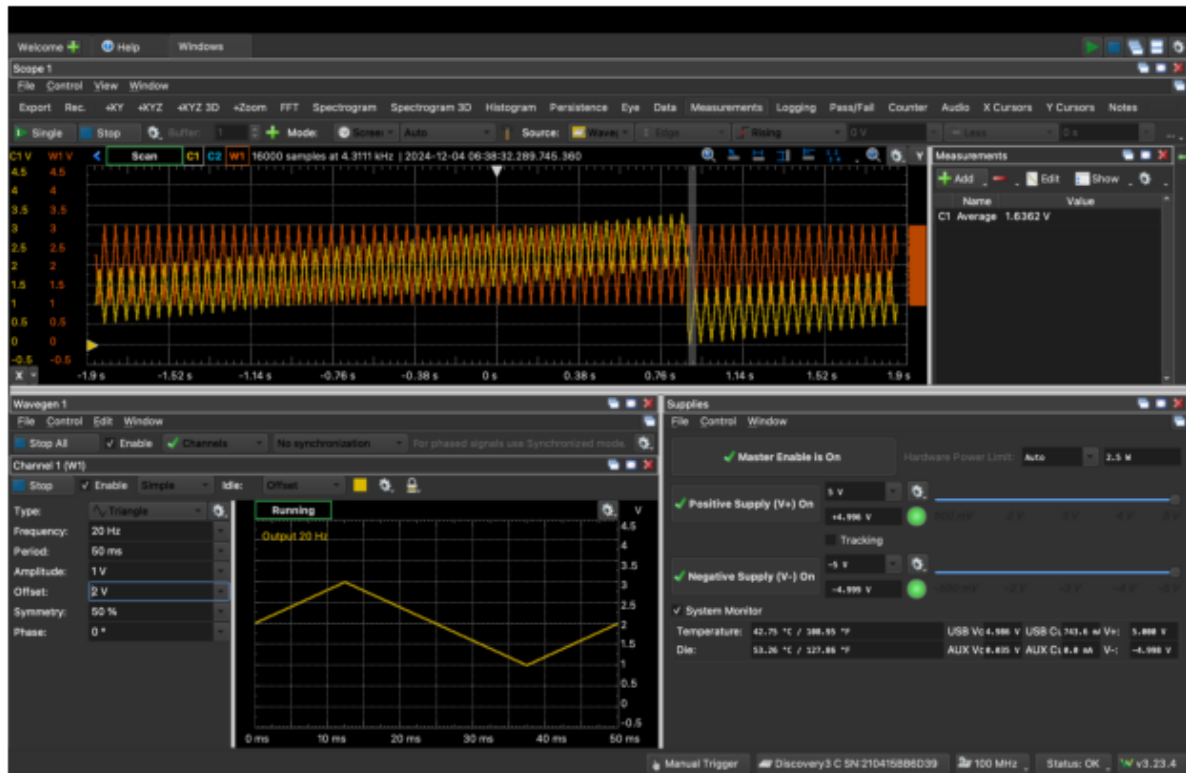


Fig. 37: Error Amplification Scope Trace

See how the average value of the input triangle wave (the offset) is below the set voltage, V^+ . According to equation (1), we see subtracted from the set voltage V^+ the integral of the input minus the set voltage over all time. Since our input's average is smaller than the set voltage V^+ , the integrand is negative and constant, so that as time goes on, the output's average increases constantly. This is the effect of the integrator error amplification. The proportional correction can be noticed by the vertical inversion of the scope relative to the wavegen. That is, since, according to equation (1), we are subtracting the input minus the set voltage, when the input is above the set voltage (in this case, at the peaks of the triangle wavegen), we are subtracting a positive number from the set voltage for the output, and vice versa when the input is below the set voltage (at the troughs of the triangle wavegen). This concludes the analysis of the proportional integrator error amplification circuit, now with fixed resistor values resulting in correct reactivity of the amplifier.

Note that I chose not to invert the signal just yet, because in my testing setup, I don't

want to be so entangled with the rest of my circuit, as it is good practice to test individual components in isolation from the rest of the circuit when the purpose is to see if they work.

Phototransistor

In figure 38 is the pinout for the phototransistor that is used in combination with the motor.

```
In [40]: InlineImage('H21A1pinout.png', 7, 7, 'Fig. 38: H21A1 phototransistor pinout')
```

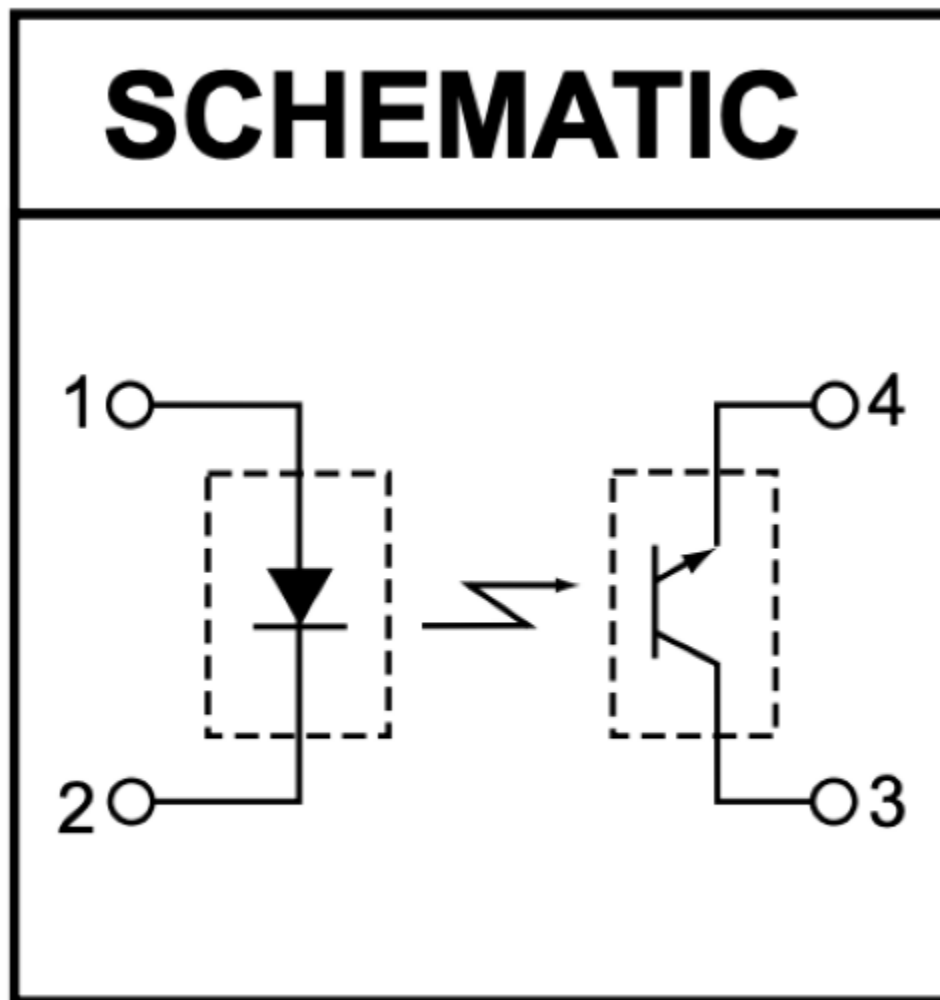


Fig. 38: H21A1 phototransistor pinout

In figure 39 below is the circuit diagram for the phototransistor setup, and in figure 40 is a testing breadboard circuit setup for it.

```
In [41]: InlineImage('photoTransTestDiag.jpeg', 7, 7, 'Fig. 39: Phototransistor Test')
```

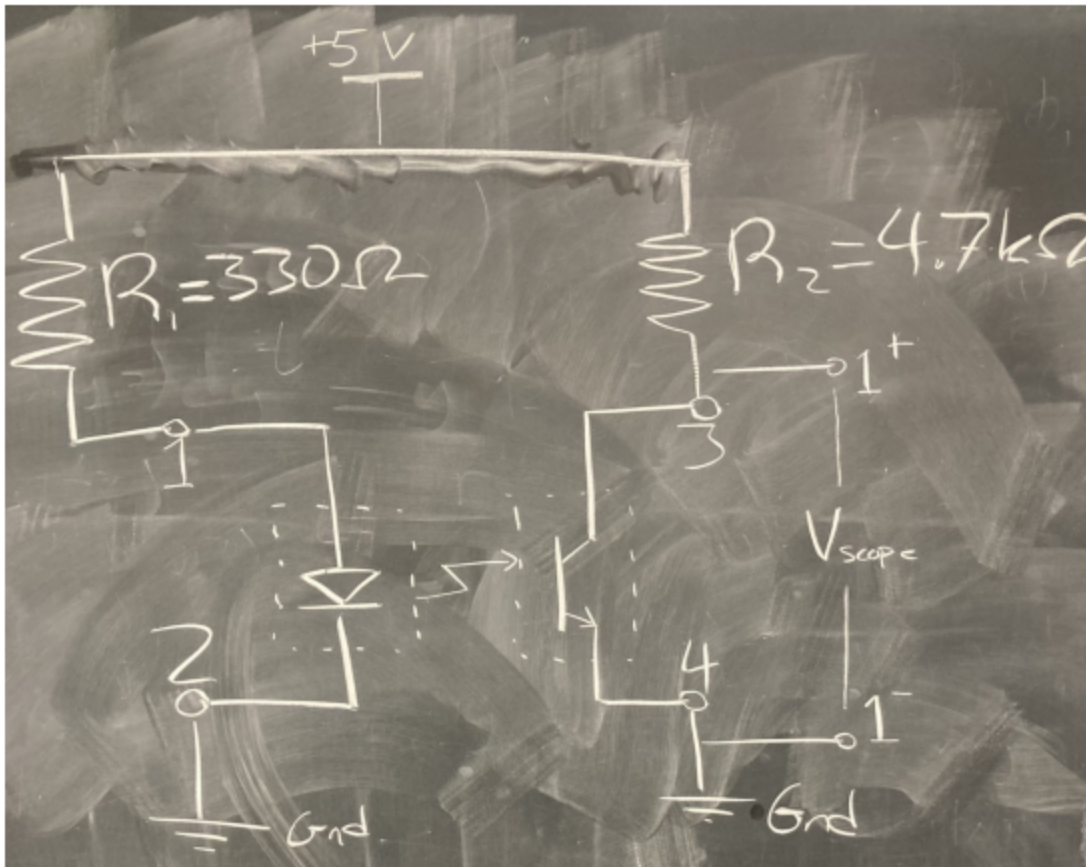



Fig. 39: Phototransistor Test Circuit Diagram

```
In [42]: InlineImage('photoTransTestCircuit.jpeg', 7, 7, 'Fig. 40: Phototransistor Te
```

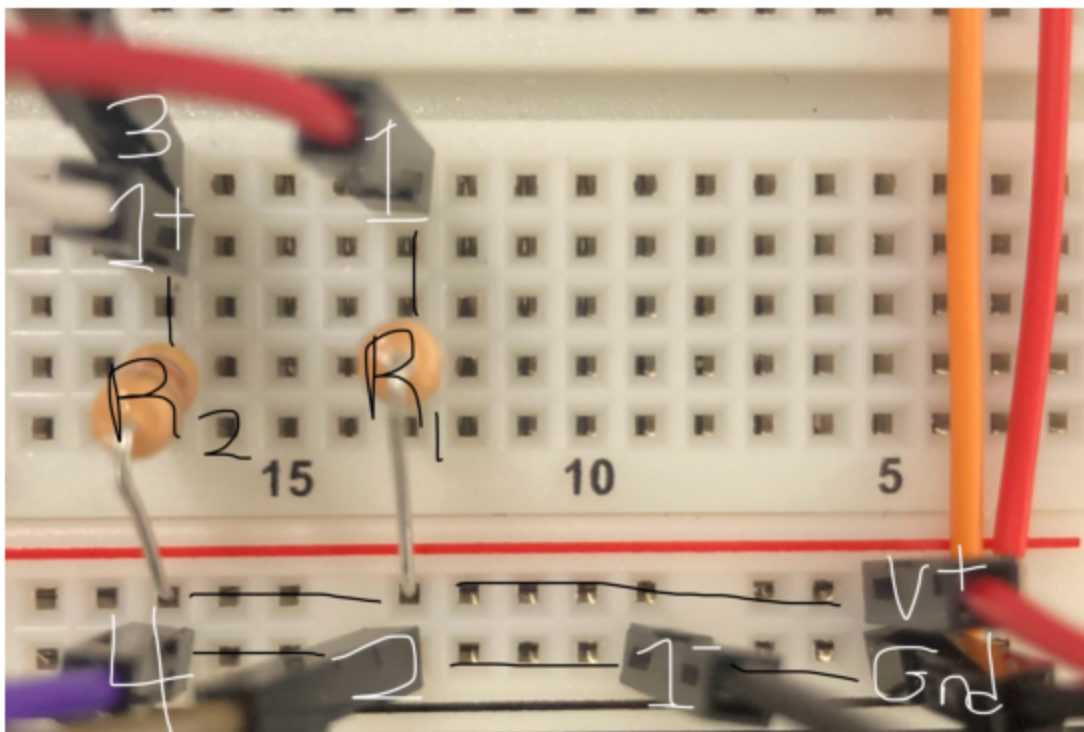


Fig. 40: Phototransistor Test Circuit

To see that it works properly, I ran a scope trace (see figure 41) with a large enough time scale that when I set the scope setting to screen, I can see the pulses pass by when I turn the wheel around. As desired, the output of the phototransistor pulses down to low when one of the holes is passed. When we invert this signal with the inverter, we will have a desirable input to our counter.

In [43]: `InlineImage('photoTransScopeFunc.png', 7, 7, 'Fig. 41: Phototransistor Scope`



Fig. 41: Phototransistor Scope Functionality

Integrating the DC Motor

TS I began to melt something (based on smell, likely the breadboard) by accident. I misread the instructions, and thought it meant simply connecting the motor in such a configuration where the positive side is connected to +5 V, negative to the resistor connected to -5 V. It turns out when I did this I was applying a voltage (which I measured, but won't measure again for the sake of documentation because the power makes the resistor quite hot) of around 5-6 volts across the resistor, making for a power of around 3 W. Considering the rating of the resistor was 3 W and when I looked online it was strongly advised against using full wattage on higher power resistors, I understood why it began to smell very quickly, and the resistor became hot very quickly. Having learned my lesson that my work should be much more scrutinized before applying higher power, I decided to use the transistor as described upon further examination in the instructions, and now it was working properly; there was a 0.3 V drop across the resistor so nothing was overheating anymore, and the motor was still running.

Seeing that my whole circuit was integrated together anyways, I decided to input the photo transistor output on the remaining inverter on the delay circuit IC, and running its output into the clock (pin 1) on the counter. Inputting a 5 Hz waveform square wave as required, as I knew everything else worked together from previous testing, I ran the square wave with all my voltage supplies setup. To test that the counting and latching section of the circuit was working in accordance with the motor, I measured my scope at the output of the DAC to see if it varied in accordance with the motor speed. To test this, I applied pressure on the motor while it was running and indeed saw that the output of the DAC was proportional to the speed of the motor.

TS I wasn't using the three prongs of the BJT, so I was seeing that the feedback circuit was useless; I applied my scope at various spots throughout the error feedback and saw that it was actually working in all places, so the issue had to be associated with the parts I just inserted: the BJT. Examining, I saw that the emitter (OR COLLECTOR) wasn't being used at all, so of course there wasn't going to be a feedback. After moving the 10 ohm resistor over one breadboard row and closing the circuit, I saw right away that the feedback was working, so the whole circuit was finally working. That being said, the motor speed control was by no means smooth; the corrections were very abrupt and way overshoot, which resulted in a motor that quickly overshoot and undershot the ideal speed (see figure 42 for faulty error correction). In the figure below (fig. 42), the scope measures the output of the DAC, and the wavegen has latch and clock frequency of 5 Hz. The speed of the motor correlated directly with the output of the scope, so that the motor did turn off at regular intervals in an overshooting effort to lower the speed. More in depth troubleshooting for the overshooting detailed above figure 37.

```
In [44]: InlineImage('wildErrorAmplification.png', 10, 10, 'Fig. 42: Error Amplificat
```



Fig. 42: Error Amplification Overshooting

Now that this is fixed, my circuit works. When I close the circuit, the motor speeds up to a certain speed set by the set voltage at the potentiometer (which I can change by turning it, hence reducing the controlled speed), and when I apply resistance to the wheel to slow it down, the motor speeds back up to the desired speed. See figures 43 and 44 showing the function of the motor at the max and a lower speed, where the latch shows the number of counts per latch, and the voltage shows the set voltage.

In [45]: `InlineImage('workingCircuit.png', 8, 8, 'Fig. 43: Lower speed working, scope`

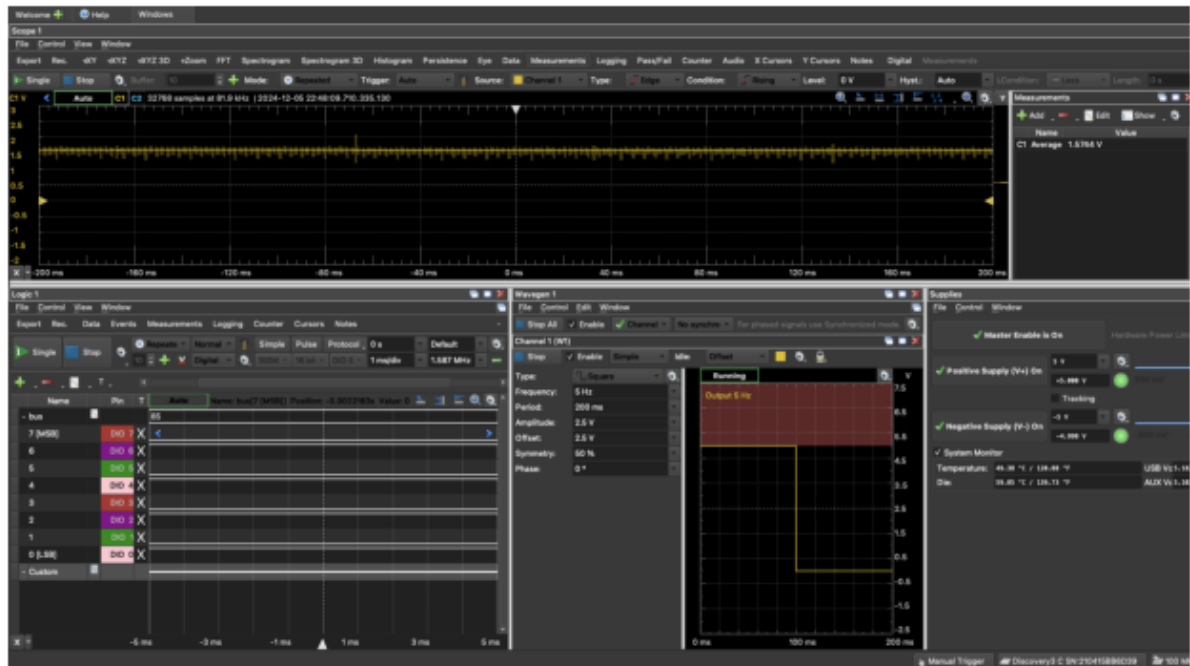


Fig. 43: Lower speed working, scope trace and logic analyzer

In [46]: `InlineImage('maxSpeed.png', 8, 8, 'Fig. 44: Maximum speed set voltage and lo`

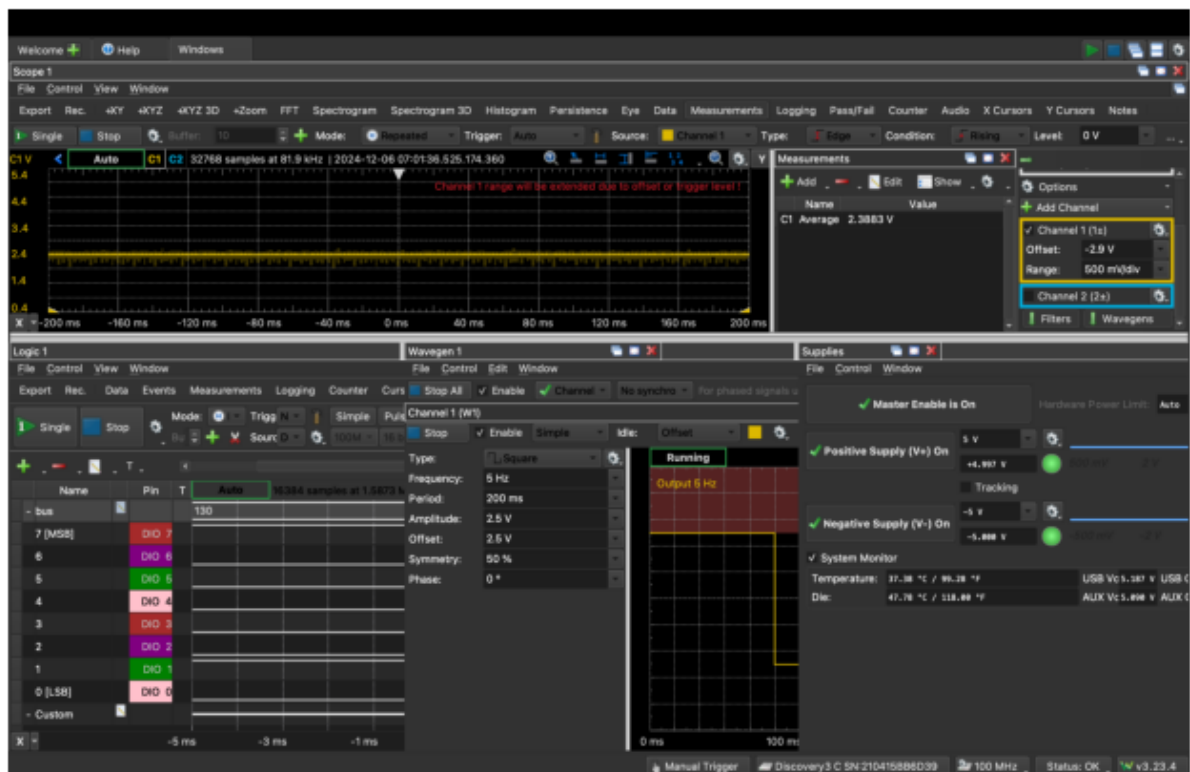


Fig. 44: Maximum speed set voltage and logic analyzer

Wire Pull:

Once he pulled the wire, the first thing I tried to do was just run the motor and see what was working. Right away I saw that the counter was still working, and when I

applied force to the motor to slow it down and let go, it sped back up to the maximum speed. So clearly my latch is working. I then checked that the buffer was working properly, and indeed it was. At this point I was relatively sure that the issue had to do with the error amplification operational amplifier, and as I was adjusting the resistance of the potentiometer, I saw that the error amplification was not listening to the set voltage; it was just maxing out no matter what. Now I'm sure that it has to do with that operational amplifier. Taking a closer look, I relatively quickly realized that the negative voltage was not connected to the Vcc- rail. This was especially easy to see since the wire is only two breadboard holes across, so the organization of my breadboard helped me out a lot here. Once I fixed this, the motor was reacting to the potentiometer set voltage once again, so the circuit was fixed.

Set Voltage to RPM relationship

In order to examine the set voltage vs. RPM relationship, I measure the scope from the output of the buffer (right before the resistor R_6) to ground, and using my logic analyzer I monitor the output of the latch. First we derive a relationship between the output of the latch and the rpm. Note that the latch output tells us how many times one of the equally spaced ten holes on the motor wheel passes by the phototransistor. Hence, 10 counts corresponds to one full rotation of the wheel. Hence, the latch divided by 10 gives us the number of rotations that occurs in 0.2 seconds ($1/f$ where f is the latch frequency), so to get the frequency (rotations per second), we would multiply this by 5. Hence, the frequency is given by one half of the latch output, and, correspondingly, the rotations per minute is given by the frequency times 60, or the latch output times 30. This in mind, since the data is discrete our uncertainty will of course be 30 times the uncertainty of the latch, which is 0.5 (since it is a discrete digital measurement). Hence, the uncertainty on the rpm is 30.

With this in mind, we can set the voltage (in my case I actually oriented myself around the latch) and see the rpm as a function of that. I alter the set voltage by turning the potentiometer, reducing the resistance so as to reduce the set voltage. I do this all the way from a latch output of 131 to 0, giving 27 datapoints, which I graphed in figure 45.

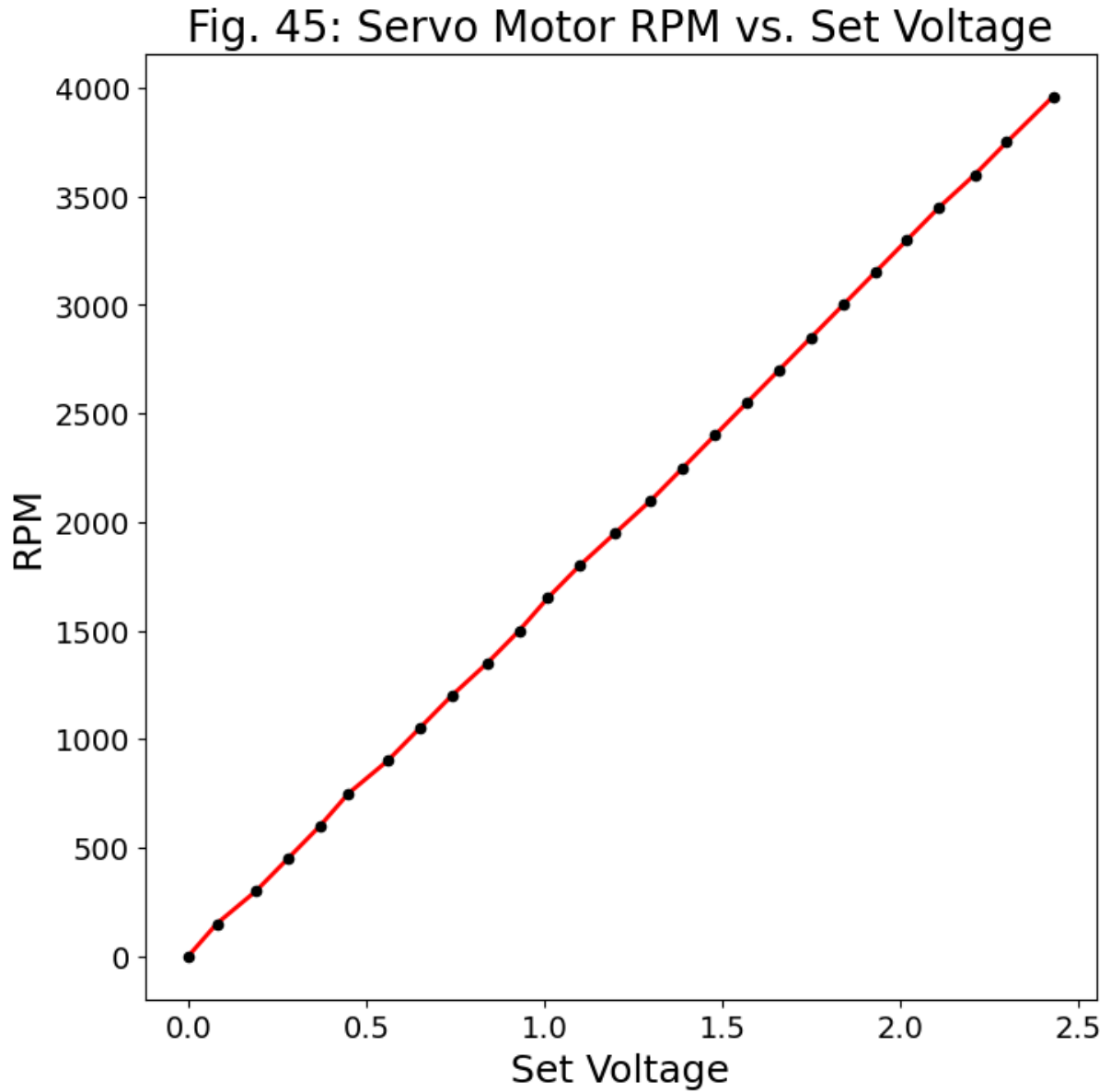
```
In [47]: import pandas as pd
import matplotlib.pyplot as plt
data = pd.read_csv('V_set vs (1).csv')
rpm = data['RPM'] #Columns in pandas dataframes can be indexed by their head
V_set = data['V_set']

fig,ax=plt.subplots(figsize=(8,8))
ax.plot(V_set, rpm, color='r', linewidth=2, zorder=1, label='Line plot')
ax.scatter(V_set, rpm, color='k', s=20, zorder=2, label='Scatter plot')

ax.tick_params(axis='x', labelsz=14)
ax.tick_params(axis='y', labelsz=14)
```

```
ax.set_xlabel('Set Voltage', fontsize=18)
ax.set_ylabel('RPM', fontsize=18)

ax.set_title('Fig. 45: Servo Motor RPM vs. Set Voltage', fontsize=20);
```



The graph above in table form:

V_set	Counts	RPM
2.43	131	3960
2.30	125	3750
2.21	120	3600
2.11	115	3450
2.02	110	3300
1.93	105	3150

V_set	Counts	RPM
1.84	100	3000
1.75	95	2850
1.66	90	2700
1.57	85	2550
1.48	80	2400
1.39	75	2250
1.30	70	2100
1.20	65	1950
1.10	60	1800
1.01	55	1650
0.93	50	1500
0.84	45	1350
0.75	40	1200
0.65	35	1050
0.56	30	900
0.45	25	750
0.37	20	600
0.28	15	450
0.19	10	300
0.08	5	150
0	0	0

We can see that the relationship can be very accurately described linear by

$$RPM = 1630 \cdot V_{set}.$$

To find the maximum voltage, I changed the set voltage to 5 volts by getting rid of the potentiometer and connecting a wire from the 5 volt rail to the input of the operational amplifier, and I saw no change in the speed of the motor. Hence, the maximum speed of the motor is around 3930 rpm, plus or minus 30.

Conclusion

In this lab, we culminated all the learnings throughout the semester into one circuit and final project, which took the form of a motor control circuit. A delay circuit using inverters and cleverly ordered RC-circuits was used to generate a 5 Hz delayed reset

pulse from an input waveform, and counters were used to count a multiple of how many times the motor rotated within each latch cycle, after which this final count was latched so that a constant voltage (converted from a binary latch output) proportional to the number of counts per latch could feed into a proportional and integrating feedback circuit using an operational amplifier, which controlled the base current into a BJT, which in turn controlled the current through, and thus the rpm, of the motor.